

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



ĐỒ ÁN LIÊN NGÀNH CÔNG NGHỆ THÔNG TIN

TÊN ĐỀ TÀI

PHÁT TRIỂN ỨNG DỤNG QUẢN LÝ CÔNG VIỆC TRÊN NỀN WEB

Giảng viên hướng dẫn: ThS.Vũ Quang Dũng

Sinh viên thực hiện: Nguyễn Văn Trường – 23010371

Lớp tín chỉ: Đồ án liên ngành – N01

Chương trình đào tạo: Hệ chính quy - Công nghệ thông tin

Hà Nội, tháng 06 năm 2026

LỜI CẢM ƠN

Em xin cảm ơn thầy Vũ Quang Dũng đã tận tình hướng dẫn, giúp đỡ em trong quá trình lên ý tưởng và thực hiện dự án. Giải đáp các thắc mắc và đưa ra hướng đi hợp lý để nhóm em có thể hoàn thành đồ án liên ngành.

LỜI CAM ĐOAN

Em cam đoan Đồ án liên ngành là sản phẩm trí tuệ của em. Mọi thông tin, dữ liệu, hình ảnh, ... được sử dụng từ các nguồn khác đều được trích dẫn đầy đủ và có thể tìm thấy các tài liệu liên quan thông qua mục tài liệu tham khảo.

Em xin chịu trách nhiệm hoàn toàn về nội dung của Đồ án liên ngành mà em đã nộp.

Hà Nội, ngày 08 tháng 06 năm 2026

Nguyễn Văn Trường

MỤC LỤC

LỜI CẢM ƠN.....	2
LỜI CAM ĐOAN.....	3
DANH MỤC HÌNH ẢNH.....	6
DANH MỤC BẢNG BIỂU.....	7
CHƯƠNG I: TỔNG QUAN ĐỀ TÀI	8
1.1 Lý do chọn đề tài.....	8
1.2 Mục tiêu và phạm vi nghiên cứu.....	9
1.3 Đối tượng và phương pháp nghiên cứu.....	9
1.4 Ý nghĩa khoa học và thực tiễn.....	10
1.5 Bố cục đồ án.....	11
1.6 Kết luận chương	12
CHƯƠNG II: CƠ SỞ LÝ THUYẾT.....	12
2.1 Tổng quan lĩnh vực	12
2.2 Các khái niệm và mô hình liên quan.....	13
2.3 Các công trình, nghiên cứu liên quan.....	14
2.4 Công nghệ, ngôn ngữ và công cụ sử dụng	15
2.5 Kết luận chương	16
CHƯƠNG III: PHÂN TÍCH HỆ THỐNG.....	17
3.1 Khảo sát hiện trạng.....	17
3.2 Phân tích yêu cầu.....	17
3.3 Mô hình Use Case sử dụng	19
3.4 Các biểu đồ phân tích.....	30
3.5 Kết luận chương	44
CHƯƠNG IV: THIẾT KẾ HỆ THỐNG	45

4.1	Kiến trúc tổng thể.....	45
4.2	Thiết kế cơ sở dữ liệu.....	46
4.3	Thiết kế thành phần phần mềm.....	51
4.4	Thiết kế giao diện người dùng.....	54
4.5	Thiết kế xử lý.....	60
4.6	Kết luận chương.....	61
CHƯƠNG V: KIỂM THỬ HỆ THỐNG.....		62
5.1	Mục tiêu kiểm thử.....	62
5.2	Chiến lược kiểm thử.....	63
5.3	Môi trường kiểm thử.....	64
5.4	Kịch bản kiểm thử.....	65
5.5	Kết luận chương.....	67
CHƯƠNG VI: CÀI ĐẶT VÀ KẾT QUẢ.....		68
6.1	Môi trường triển khai.....	68
6.2	Cài đặt các chức năng chính.....	69
6.4	Đánh giá hệ thống.....	70
TÀI LIỆU THAM KHẢO.....		72

DANH MỤC HÌNH ẢNH

Hình 3.1: Use Case xác thực	19
Hình 3.2: Use case quản lý workspace	21
Hình 3.3: Use case quản lý công việc	22
Hình 3.4: Use case quản lý ghi chú và nhắc nhở	24
Hình 3.5: Use case quản lý tài liệu	27
Hình 3.11: Activity Diagram Đăng ký/Đăng nhập	30
Hình 3.12: Activity Diagram mở workspace	31
Hình 3.13: Activity Diagram quản lý công việc	33
Hình 3.14: Activity Diagram quản lý bài tập	35
Hình 3.15: Activity Diagram Browser Extension	36
Hình 3.17: Sequence Diagram Đăng ký	38
Hình 3.18: Sequence Diagram Đăng nhập	39
Hình 3.19: Sequence Diagram mở Workspace	40
Hình 3.20: Sequence Diagram lưu workspace từ extension	41
Hình 3.21: Sequence Diagram quản lý công việc	42
Hình 3.22: Sequence Diagram ghi chú và nhắc nhở	43
Hình 4.1: Sơ đồ ERD	47
Hình 4.2: Giao diện Dashboard	55
Hình 4.3: Giao diện Lịch trình	55
Hình 4.4: Giao diện nhắc nhở	56
Hình 4.5: Giao diện quản lý workspace	56
Hình 4.6: Giao diện quản lý công việc	57
Hình 4.7: Giao diện ghi chú	57

DANH MỤC BẢNG BIỂU

Bảng 4.1: Bảng dữ liệu User	48
Bảng 4.2: Bảng dữ liệu Workspace	48
Bảng 4.3: Bảng dữ liệu WorkspaceTab	49
Bảng 4.4: Bảng dữ liệu Project	49
Bảng 4.5: Bảng dữ liệu Task.....	50
Bảng 4.6: Bảng dữ liệu Note.....	50
Bảng 4.7: Bảng dữ liệu Reminder.....	51
Bảng 5.1: Kiểm thử đăng nhập	65
Bảng 5.2: Kiểm thử tạo lớp học.....	65
Bảng 5.3: Kiểm thử quản lý công việc	66
Bảng 5.4: Kiểm thử tương tác web và extension	66
Bảng 5.5: Kiểm thử bảo mật luồng API	67

CHƯƠNG I: TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong kỷ nguyên số hiện nay, sinh viên và những người làm việc tri thức (knowledge workers) phải đối mặt với một vấn đề ngày càng nghiêm trọng: Sự phân mảnh tài nguyên số (Digital Fragmentation). Một quy trình làm việc thông thường đòi hỏi người dùng phải liên tục chuyển đổi giữa hàng loạt công cụ khác nhau như ứng dụng ghi chú, phần mềm quản lý công việc (To-do list), ứng dụng chat, và hàng chục tab trình duyệt web.

Theo nhiều nghiên cứu về tương tác người-máy (Human-Computer Interaction - HCI), việc phải chuyển đổi ngữ cảnh liên tục (Context Switching) không chỉ làm giảm hiệu suất làm việc mà còn làm tăng tải trọng nhận thức (cognitive load), dẫn đến sự mệt mỏi và mất tập trung. Hầu hết các ứng dụng quản lý năng suất hiện nay (như Notion, Trello, Todoist) chỉ giải quyết được một khía cạnh của vấn đề (lưu trữ hoặc lên lịch), mà thiếu đi khả năng "điều phối không gian làm việc" một cách liền mạch ngay trên trình duyệt của người dùng.

Nhận thấy khoảng trống đó, đề tài "Xây dựng nền tảng quản lý công việc – OmniDash" được đề xuất. OmniDash không chỉ là một ứng dụng To-do list thông thường, mà là một "Hệ điều hành năng suất" (Productivity OS) thu nhỏ. Điểm đột phá của đề tài là khái niệm "Workspace Combos" – cho phép người dùng đóng gói các tài nguyên (URLs, task, note) theo từng ngữ cảnh và kích hoạt chúng tức thì thông qua sự kết hợp giữa Web Application và Browser Extension. Giải pháp này giúp người dùng loại bỏ thao tác thủ công, giảm thiểu sự phân tán thông tin và nhanh chóng bước vào trạng thái tập trung sâu (Deep Work).

1.2 Mục tiêu và phạm vi nghiên cứu

1.2.1 Mục tiêu nghiên cứu

- Phân tích, thiết kế và xây dựng thành công hệ thống quản lý công việc OmniDash.
- Thiết kế kiến trúc hệ thống Layered Monolithic Architecture đảm bảo tính mở rộng, khả năng bảo trì và hiệu năng.
- Xây dựng Web Application quản trị tập trung (Công việc, Ghi chú, Workspace) với giao diện người dùng tối giản..
- Phát triển Browser Extension (Tiện ích mở rộng) có khả năng giao tiếp bảo mật với Web App để thực thi việc điều khiển và quản lý Tab trình duyệt..

1.2.2 Phạm vi nghiên cứu

Đề tài tập trung vào:

- Phát triển ứng dụng web sử dụng frontend Next.js và backend Node.js/Express.
- Hệ quản trị cơ sở dữ liệu PostgreSQL, giao tiếp thông qua Prisma ORM.
- Lập trình Extension theo chuẩn Manifest V3 cho các trình duyệt nhân Chromium.

Ngoài phạm vi:

- Không triển khai mobile app native.
- Không đi sâu vào tối ưu hệ thống ở quy mô doanh nghiệp lớn.
- Không nghiên cứu AI hoặc phân tích dữ liệu nâng cao.

1.3 Đối tượng và phương pháp nghiên cứu

1.3.1 Đối tượng nghiên cứu

- Quy trình quản lý công việc cá nhân, phương pháp tối ưu hóa sự tập trung (thông qua Context Switching), và kỹ thuật giao tiếp giữa ứng dụng Web với trình duyệt (Web-to-Extension Communication).
- Công nghệ phát triển web hiện đại gồm:
 - Frontend: Next.js, Ant Design, Axios.
 - Backend: Node.js, Express.js.
 - Database: PostgreSQL.
 - Extension: Manifest V3

1.3.2 Phương pháp nghiên cứu

- Phương pháp lý thuyết: Tìm hiểu các tài liệu, bài báo về kiến trúc phần mềm (đặc biệt là Layered Monolithic Architecture) và các công nghệ phát triển Web hiện đại.
- Phương pháp thực nghiệm: Phân tích yêu cầu, mô hình hóa hệ thống bằng UML, lập trình xây dựng sản phẩm thực tế, sau đó tiến hành kiểm thử để đánh giá mức độ đáp ứng yêu cầu (đặc biệt là yêu cầu về thời gian phản hồi $< 1.5s$).

1.4 Ý nghĩa khoa học và thực tiễn

1.4.1 Ý nghĩa khoa học

- Góp phần nghiên cứu và áp dụng mô hình phát triển ứng dụng web hiện đại.

- Minh họa việc kết hợp giữa REST API và giao tiếp với extension trong cùng hệ thống.
- Ứng dụng mô hình client-server và quản lý dữ liệu quan hệ trong xây dựng phần mềm.

1.4.2 Ý nghĩa thực tiễn

- Hỗ trợ sinh viên hoặc nhóm làm việc quản lý công việc hiệu quả hơn.
- Giúp theo dõi tiến độ, tập trung tài liệu và công cụ.
- Tạo nền tảng quản lý công việc hiệu quả và giảm thiểu các thao tác thủ công khi thiết lập môi trường làm việc trên máy tính.

1.5 Bố cục đồ án

Nội dung đồ án được chia thành các chương như sau:

- **Chương I:** Tổng quan đề tài - trình bày lý do chọn đề tài, mục tiêu, phạm vi và phương pháp nghiên cứu.
- **Chương II:** Cơ sở lý thuyết - giới thiệu các công nghệ và kiến thức nền tảng sử dụng trong hệ thống.
- **Chương III:** Phân tích hệ thống - mô tả yêu cầu, kiến trúc, cơ sở dữ liệu và luồng xử lý.
- **Chương IV:** Thiết kế hệ thống - trình bày quá trình phát triển, chức năng và giao diện hệ thống.
- **Chương V:** Kiểm thử hệ thống - đánh giá hiệu năng, tính ổn định và kết quả đạt được.
- **Chương VI:** Cài đặt và kết quả - tổng kết và đề xuất cải tiến trong tương lai.

1.6 Kết luận chương

Chương I đã trình bày tổng quan về đề tài xây dựng hệ thống quản lý công việc, bao gồm lý do lựa chọn, mục tiêu, phạm vi nghiên cứu, phương pháp thực hiện cũng như ý nghĩa khoa học và thực tiễn của đề tài. Đây là cơ sở để triển khai các nội dung tiếp theo như cơ sở lý thuyết, phân tích thiết kế và xây dựng hệ thống trong các chương sau.

CHƯƠNG II: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan lĩnh vực

Trong thập kỷ qua, lĩnh vực phần mềm quản lý công việc (Task Management Software) đã chứng kiến sự phát triển bùng nổ nhằm đáp ứng nhu cầu tổ chức thông tin của người dùng cá nhân và doanh nghiệp. Từ những ứng dụng ghi chú đơn giản ban đầu, lĩnh vực này đã tiến hóa thành các nền tảng số hóa phức tạp.

Tuy nhiên, bối cảnh làm việc hiện đại đặt ra một thách thức mới: sự phân tán của tài nguyên số. Một công việc cụ thể không chỉ tồn tại dưới dạng một dòng văn bản (to-do item) mà còn gắn liền với nhiều tài nguyên khác nhau như tài liệu trực tuyến, công cụ giao tiếp và các trang web nghiệp vụ. Việc quản lý công việc hiện nay không chỉ dừng lại ở việc "ghi nhớ" mà đòi hỏi khả năng "thiết lập môi trường" nhanh chóng. Lĩnh vực này đang chuyển dịch từ việc tạo ra các công cụ lưu trữ thụ động sang các hệ thống chủ động hỗ trợ người dùng tự động hóa quy trình chuyển đổi ngữ cảnh làm việc ngay trên trình duyệt máy tính.

2.2 Các khái niệm và mô hình liên quan

2.2.1 Mô hình kiến trúc Client–Server

Mô hình client–server là mô hình phổ biến trong phát triển ứng dụng web, trong đó:

- Client (Frontend): giao diện tương tác với người dùng, gửi yêu cầu và hiển thị dữ liệu.
- Server (Backend): xử lý nghiệp vụ, xác thực người dùng và thao tác cơ sở dữ liệu.

Mô hình này giúp hệ thống dễ mở rộng, bảo trì và phân tách rõ ràng giữa giao diện và xử lý dữ liệu.

2.2.2 RESTful API

RESTful API là kiến trúc giao tiếp giữa client và server thông qua giao thức HTTP với các phương thức cơ bản như GET, POST, PUT và DELETE. Việc áp dụng RESTful API giúp:

- Dễ dàng tích hợp với nhiều nền tảng.
- Chuẩn hóa cách truyền dữ liệu.
- Tăng khả năng mở rộng của hệ thống.

2.2.3 Mô hình Kiến trúc Phân lớp

Để xây dựng một hệ thống phần mềm quản lý công việc vững chắc, đề tài áp dụng mô hình Kiến trúc Phân lớp (Layered Monolithic Architecture). Mô hình này

chia hệ thống thành các tầng độc lập, tuân thủ quy tắc phụ thuộc một chiều từ trên xuống, bao gồm:

- **Presentation Layer (Tầng trình bày):** Xử lý các yêu cầu HTTP, xác thực người dùng và kết xuất giao diện. Tầng này chứa các Controller nhận dữ liệu từ Web và Extension.
- **Business Logic Layer (Tầng nghiệp vụ):** Chứa các quy tắc nghiệp vụ cốt lõi, xác thực tính hợp lệ và điều phối việc kết nối dữ liệu.
- **Persistence Layer (Tầng truy cập dữ liệu):** Đảm nhiệm việc ánh xạ các đối tượng nghiệp vụ vào cơ sở dữ liệu và thực thi các thao tác CRUD.
- **Data Layer (Tầng dữ liệu):** Hệ thống lưu trữ cơ sở dữ liệu vật lý.

2.2.4 Cơ sở dữ liệu

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ, lưu trữ dữ liệu dưới dạng bảng với các hàng và cột. PostgreSQL phù hợp với các ứng dụng web hiện đại nhờ

- Hỗ trợ chuẩn SQL và đảm bảo tính toàn vẹn dữ liệu cao.
- Khả năng mở rộng tốt và xử lý hiệu quả khối lượng dữ liệu lớn.
- Hỗ trợ nhiều tính năng nâng cao như transaction, indexing, view và stored procedure.
- Hiệu năng ổn định, độ tin cậy cao và khả năng bảo mật tốt.

2.3 Các công trình, nghiên cứu liên quan

Hiện nay, trên thị trường có nhiều giải pháp phần mềm quản lý công việc nổi bật như Trello, Todoist, hay Notion. Các công trình này cung cấp các mô hình

quản lý tác vụ ưu việt như Kanban (bảng trạng thái) hay List (danh sách truyền thống).

Tuy nhiên, qua quá trình nghiên cứu và đối chiếu, các hệ thống trên đều tồn tại một giới hạn chung: chúng hoạt động độc lập với môi trường trình duyệt của người dùng. Nghĩa là, Notion có thể nhắc nhở người dùng về việc "Lập trình chức năng A", nhưng không thể tự động mở các tab Github, StackOverflow hay tài liệu API liên quan đến chức năng đó. Hệ thống OmniDash kế thừa mô hình quản lý tác vụ của các công trình đi trước, đồng thời khắc phục nhược điểm này bằng cách kết hợp với Tiện ích mở rộng trình duyệt (Browser Extension) để tạo ra khái niệm "Workspace Combos" – liên kết chặt chẽ giữa công việc và tài nguyên web.

2.4 Công nghệ, ngôn ngữ và công cụ sử dụng

2.4.1 Frontend

Next.js là một framework của React hỗ trợ tối ưu hóa kết xuất giao diện (Rendering), kết hợp cùng thư viện Tailwind CSS giúp xây dựng giao diện người dùng (UI) tối giản, phản hồi nhanh và dễ dàng tùy biến giao diện theo từng ngữ cảnh làm việc.

2.4.2 Backend

Express.js và Node.js : Môi trường thực thi mạnh mẽ với cơ chế Non-blocking I/O, giúp xây dựng hệ thống RESTful API tại tầng Presentation có khả năng xử lý nhanh chóng các yêu cầu đồng bộ hóa dữ liệu từ Browser Extension.

2.4.3 Cơ sở dữ liệu

PostgreSQL và Prisma ORM (Database): PostgreSQL đảm bảo tính toàn vẹn và an toàn cho dữ liệu người dùng. Prisma ORM đóng vai trò là tầng Persistence, giúp tương tác với cơ sở dữ liệu thông qua các đối tượng (Objects) thay vì câu lệnh SQL thuần túy, tăng tốc độ phát triển và giảm thiểu rủi ro bảo mật (như SQL Injection).

2.4.4 Extension

Manifest V3 (Chrome Extension API): Chuẩn phát triển tiện ích mở rộng mới nhất của Google, cung cấp các hàm API an toàn để tương tác với trình duyệt, cho phép hệ thống điều khiển việc đóng/mở hàng loạt các Tab theo lệnh từ ứng dụng Web.

2.4.5 Công cụ hỗ trợ phát triển

- Visual Studio Code: môi trường lập trình chính.
- Git và GitHub: quản lý phiên bản mã nguồn.
- Postman: kiểm thử API.
- npm: quản lý package và dependency.

2.5 Kết luận chương

Chương II đã trình bày các cơ sở lý thuyết làm nền tảng cho việc xây dựng hệ thống quản lý dự án, bao gồm tổng quan lĩnh vực, các khái niệm và mô hình kiến trúc liên quan, các công trình tham khảo cũng như các công nghệ và công cụ được sử dụng trong quá trình phát triển. Những kiến thức này là cơ sở quan trọng để phân tích và thiết kế hệ thống ở chương tiếp theo.

CHƯƠNG III: PHÂN TÍCH HỆ THỐNG

3.1 Khảo sát hiện trạng

Qua quá trình khảo sát thói quen làm việc trên môi trường số của sinh viên và người làm việc độc lập (freelancer), phần lớn người dùng đang sử dụng song song từ 2 đến 3 công cụ khác nhau. Ví dụ: sử dụng Todoist để ghi chú công việc, kết hợp với các dấu trang (bookmarks) trên trình duyệt để lưu trữ tài liệu.

Vấn đề cốt lõi phát sinh là sự đứt gãy trong luồng công việc. Khi người dùng muốn bắt đầu một tác vụ (ví dụ: "Làm bài tập lập trình Web"), họ phải thao tác thủ công: mở trình duyệt, gõ địa chỉ Github, tìm tài liệu ReactJS, mở trang hướng dẫn, và quay lại ứng dụng quản lý tác vụ để đánh dấu đang thực hiện. Quy trình này lặp đi lặp lại gây mất thời gian và dễ làm người dùng xao nhãng bởi các tab không liên quan (như mạng xã hội) còn sót lại từ phiên làm việc trước. Từ hiện trạng này, yêu cầu cấp thiết là phải có một hệ thống có khả năng tự động hóa việc mở/đóng các tài nguyên web theo đúng ngữ cảnh của công việc đang thực hiện.

3.2 Phân tích yêu cầu

3.2.1 Yêu cầu chức năng

Dựa trên kết quả khảo sát, hệ thống OmniDash cần đáp ứng các chức năng cốt lõi sau:

Quản lý tài khoản

- Đăng ký, đăng nhập và quản lý phiên hoạt động một cách an toàn bằng JSON Web Token (JWT)
- Quản lý thông tin cá nhân.

- Phân quyền người dùng.

Quản lý công việc

- Tạo, cập nhật và xóa công việc.
- Thêm các việc làm con.
- Phân loại công việc theo mức độ, theo dõi tiến độ.

Quản lý không gian làm việc

- Thêm, sửa, xóa workspace.
- Gán công việc và ghi chú cho workspace..

Quản lý ghi chú

- Thêm, xóa, sửa ghi chú
- Gán ghi chú cho workspace hoặc website(với extension).
- Ghim và tìm kiếm ghi chú

3.2.2 Yêu cầu phi chức năng

- Thời gian phản hồi kể từ khi người dùng bấm "Kích hoạt" đến khi trình duyệt bắt đầu mở các Tab mới phải dưới 1.5 giây.
- Toàn bộ API giao tiếp giữa Web App, Browser Extension và Database phải được mã hóa qua giao thức HTTPS. Mật khẩu người dùng phải được băm (hashing) bằng bcrypt trước khi lưu vào cơ sở dữ liệu.
- Giao diện (UI) thiết kế theo phong cách tối giản, có khả năng phản hồi (Responsive) trên nhiều kích thước màn hình máy tính. Thời gian chuyển

đổi giao diện giữa các chế độ (Context Switching) diễn ra mượt mà dưới 500ms.

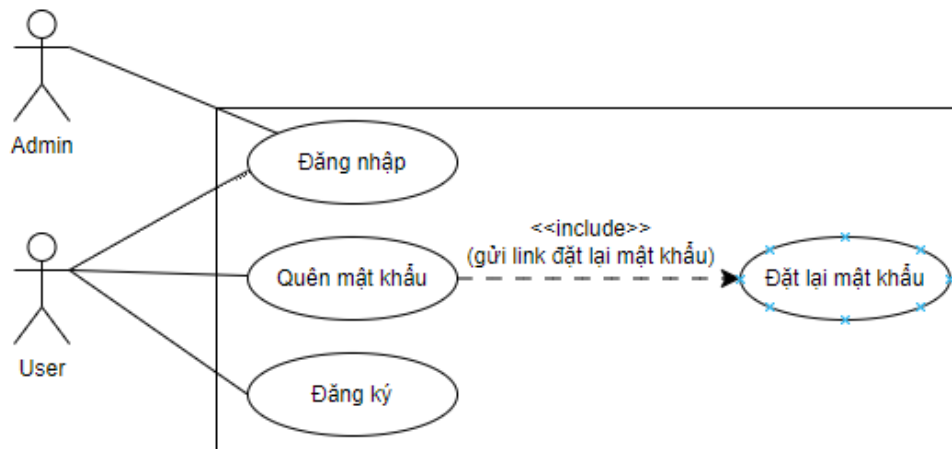
3.3 Mô hình Use Case sử dụng

3.3.1 Danh sách Use Case

- Use Case xác thực
- Use Case quản lý workspace
- Use Case quản lý công việc
- Use Case quản lý ghi chú và nhắc nhở
- Use Case Administration

3.3.2 Mô tả chi tiết từng Use Case

- Use Case xác thực



Hình 3.1: Use Case xác thực

Tác nhân chính: User, Admin

Mục tiêu: Cho phép người dùng đăng ký tài khoản, đăng nhập hệ thống và khôi phục mật khẩu một cách an toàn.

Mô tả tóm tắt: Use case này mô tả các chức năng xác thực người dùng trong hệ thống, bao gồm đăng ký tài khoản, đăng nhập khôi phục mật khẩu thông qua liên kết gửi qua email.

Luồng sự kiện chính:

- Người dùng chọn đăng ký, đăng nhập hoặc quên mật khẩu.
- Khi đăng ký, hệ thống email(bắt buộc), tên người dùng và mật khẩu.
- Khi đăng nhập, hệ thống yêu cầu cung cấp email và mật khẩu.
- Khi quên mật khẩu, hệ thống gửi liên kết đặt lại mật khẩu qua email để người dùng cập nhật mật khẩu mới.

Luồng phụ:

- Email xác nhận hoặc liên kết đặt lại mật khẩu không hợp lệ hoặc hết hạn.
- Người dùng nhập sai thông tin đăng nhập.

Điều kiện tiên quyết:

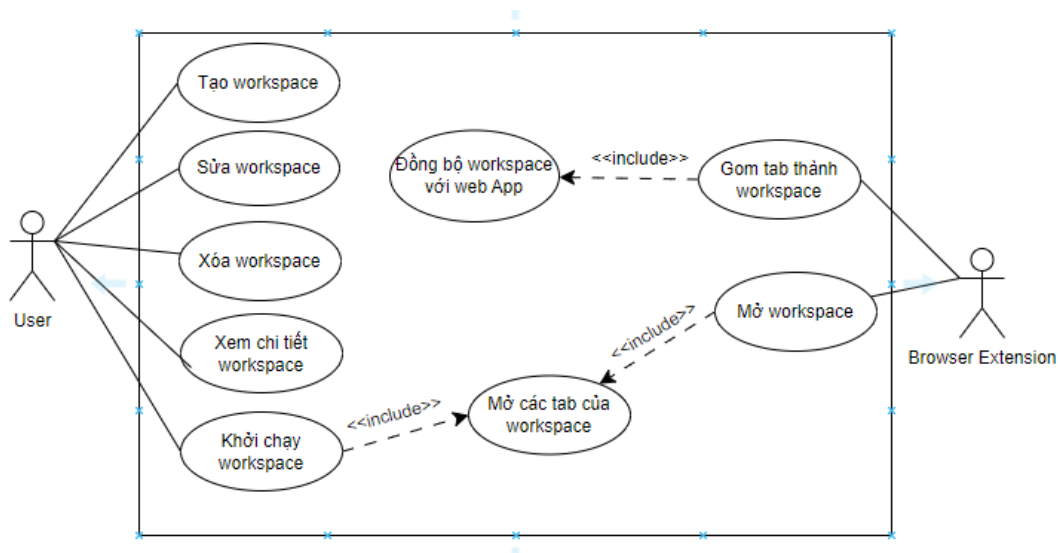
- Người dùng có kết nối Internet.
- Hệ thống hoạt động bình thường.

Điều kiện sau khi hoàn tất:

- Người dùng đăng nhập thành công hoặc tài khoản được tạo mới.

- Mật khẩu được cập nhật thành công nếu thực hiện khôi phục.

- **Use Case quản lý workspace**



Hình 3.2: Use case quản lý workspace

Tác nhân chính: User, Browser Extension

Mục tiêu: Hỗ trợ người dùng sử dụng và quản lý các workspace ở cả web app và extension. Qua đó giúp linh hoạt trong việc sử dụng và gia tăng năng suất làm việc.

Mô tả tóm tắt: Use case này mô tả các chức năng quản lý workspace trong hệ thống ở cả web app và browser extension. Giúp người dùng thêm, xóa, sửa và khởi chạy workspace ở cả hai nơi. Với sự đồng bộ dữ liệu sẽ giúp quá trình trải nghiệm được nhất quán.

Luồng sự kiện:

- Người dùng thêm, xóa, sửa workspace.
- Người dùng khởi chạy workspace sau đó chỉnh duyệt sẽ mở tất cả các tab thuộc workspace đó.
- Sử dụng extension để gom các tab tạo thành workspace sau đó đồng bộ dữ liệu với web app.
- Người dùng mở workspace ngay trên extension mà không cần truy cập web app.

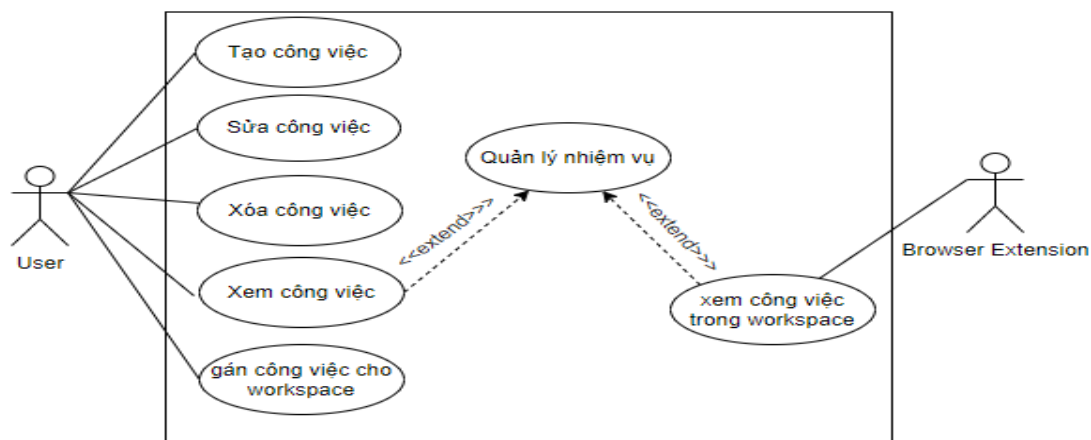
Điều kiện tiên quyết:

- Người dùng đã đăng nhập vào hệ thống.
- Người dùng đã cài extension trên trình duyệt.

Điều kiện sau khi hoàn tất:

- Người dùng thành công mở tất cả các tab trong workspace.
- Người dùng quản lý được các workspace của mình.

• Use Case quản lý Công việc



Hình 3.3: Use case quản lý công việc

Tác nhân chính: User, browser extension

Mục tiêu: Hỗ trợ người dùng tạo và quản lý công việc. Bằng cách sử dụng các chức năng của web và extension giúp người dùng theo dõi, quản lý từ đó tăng năng suất làm việc.

Mô tả tóm tắt: Use case này mô tả các chức năng quản lý công việc trong hệ thống quản lý công việc. Người dùng có thể xem danh sách công việc, xem chi tiết từng công việc, tạo mới, cập nhật trạng thái, chỉnh sửa hoặc xóa công việc khi cần thiết. Ngoài ra, người dùng có thể theo dõi tiến độ thực hiện, thời hạn hoàn thành và quản lý các nhiệm vụ liên quan nhằm đảm bảo công việc được thực hiện hiệu quả.

Luồng sự kiện:

- Người dùng thêm, xóa, sửa công việc.
- Người dùng ấn vào xem chi tiết công việc và thực hiện các thao tác để quản lý các nhiệm vụ trong đó.
- Gán các công việc cho workspace mong muốn.
- Người dùng quản lý các nhiệm vụ trong công việc được gán vào workspace ở extension

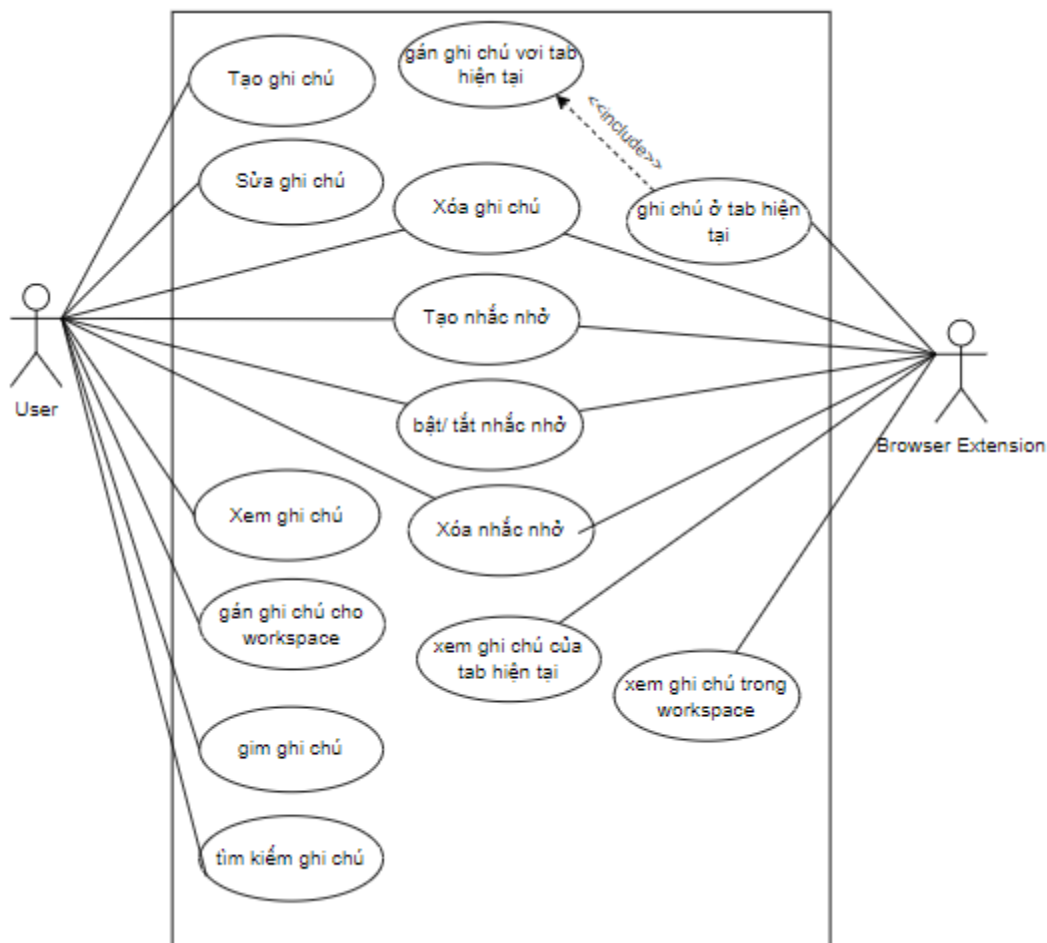
Điều kiện tiên quyết:

- Người dùng đã đăng nhập vào hệ thống.
- Trình duyệt đã cài đặt extension

Điều kiện sau khi hoàn tất:

- Người dùng quản lý được công việc và các nhiệm vụ trong đó.
- Người dùng quản lý linh hoạt hơn qua việc sử dụng extension và gắn workspace.

- **Use Case quản lý ghi chú và nhắc nhở**



Hình 3.4: Use case quản lý ghi chú và nhắc nhở

Tác nhân chính: User, browser extension

Mục tiêu: Hỗ trợ người dùng tạo, quản lý ghi chú và thiết lập nhắc nhở liên kết với tab trình duyệt hoặc workspace, giúp lưu trữ thông tin, theo dõi công việc và nhận thông báo đúng thời điểm.

Mô tả tóm tắt: Use case này mô tả các chức năng quản lý ghi chú và nhắc nhở trong hệ thống. Người dùng có thể tạo, xem, sửa, ghim, tìm kiếm và xóa ghi chú. Ngoài ra, người dùng có thể gắn ghi chú với tab hiện tại hoặc workspace. Đối với nhắc nhở, người dùng có thể tạo, bật/tắt và xóa nhắc nhở cho các ghi chú hoặc công việc quan trọng. Browser Extension hỗ trợ hiển thị ghi chú theo tab hiện tại hoặc workspace đang sử dụng và thực hiện các thông báo nhắc nhở khi cần thiết.

Luồng sự kiện chính:

- Người dùng tạo ghi chú mới.
- Người dùng gắn ghi chú với tab hiện tại hoặc workspace.
- Hệ thống lưu ghi chú vào cơ sở dữ liệu.
- Người dùng xem danh sách ghi chú.
- Người dùng xem chi tiết nội dung ghi chú.
- Người dùng chỉnh sửa nội dung ghi chú.
- Người dùng ghim ghi chú quan trọng.
- Người dùng tìm kiếm ghi chú theo từ khóa.
- Người dùng tạo nhắc nhở cho ghi chú hoặc công việc.
- Người dùng bật hoặc tắt nhắc nhở.
- Browser Extension hiển thị ghi chú liên kết với tab hiện tại.
- Browser Extension hiển thị các ghi chú thuộc workspace hiện tại.
- Khi đến thời gian đã thiết lập, Browser Extension gửi thông báo nhắc nhở cho người dùng.

- Người dùng xóa ghi chú hoặc xóa nhắc nhở khi không còn nhu cầu sử dụng.

Luồng phụ:

- Người dùng gắn ghi chú với tab hiện tại khi tạo hoặc chỉnh sửa ghi chú.
- Người dùng gắn ghi chú với workspace để quản lý theo nhóm công việc.
- Browser Extension tự động hiển thị ghi chú khi người dùng mở tab đã được liên kết.
- Người dùng nhập từ khóa tìm kiếm nhưng không tìm thấy ghi chú phù hợp.
- Người dùng tạo nhắc nhở nhưng không nhập thời gian hợp lệ.
- Người dùng bật hoặc tắt nhắc nhở nhiều lần trước thời điểm thông báo.
- Ghi chú hoặc nhắc nhở được chọn để chỉnh sửa hoặc xóa không tồn tại.
- Browser Extension không thể truy cập thông tin tab hiện tại do thiếu quyền truy cập.
- Browser Extension không thể gửi thông báo nhắc nhở do lỗi kết nối hoặc người dùng chưa cấp quyền thông báo.

Điều kiện tiên quyết:

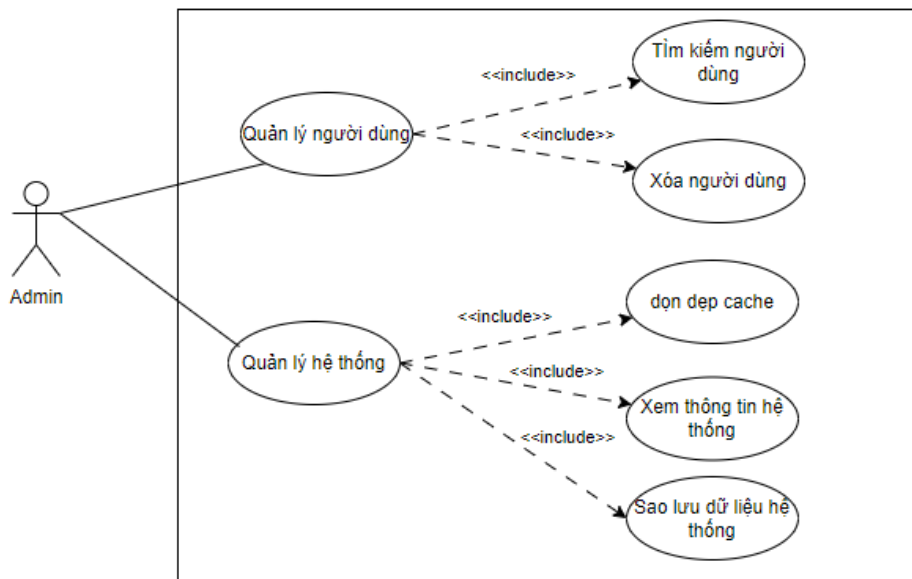
- Người dùng đã đăng nhập vào hệ thống.
- Browser Extension đã được cài đặt và cấp các quyền cần thiết.
- Workspace hoặc tab trình duyệt tồn tại trong hệ thống.
- Thiết bị của người dùng cho phép nhận thông báo từ Browser Extension.

Điều kiện sau khi hoàn tất:

- Ghi chú được tạo, cập nhật, ghim hoặc xóa thành công.
- Nhắc nhở được tạo, bật/tắt hoặc xóa thành công.
- Ghi chú được liên kết với tab hoặc workspace tương ứng.

- Browser Extension hiển thị đúng các ghi chú liên quan đến tab hoặc workspace hiện tại.
- Người dùng nhận được thông báo khi đến thời gian nhắc nhở.
- Dữ liệu ghi chú và nhắc nhở được cập nhật trong hệ thống.

- **Use Case quản trị viên**



Hình 3.5: Use case quản lý tài liệu

Tác nhân chính: Admin

Mục tiêu: Hỗ trợ quản trị viên quản lý người dùng và hệ thống nhằm đảm bảo hệ thống hoạt động ổn định, an toàn và hiệu quả.

Mô tả tóm tắt: Use case này mô tả các chức năng quản lý người dùng và quản lý hệ thống trong hệ thống. Quản trị viên có thể tìm kiếm và xóa người dùng. Ngoài ra, quản trị viên có thể thực hiện các thao tác quản lý hệ thống như xem thông tin

hệ thống, dọn dẹp bộ nhớ đệm (cache) và sao lưu dữ liệu hệ thống để đảm bảo hiệu suất và tính an toàn của dữ liệu.

Luồng sự kiện chính:

- Quản trị viên truy cập chức năng quản lý người dùng.
- Quản trị viên tìm kiếm người dùng cần quản lý.
- Hệ thống hiển thị thông tin người dùng phù hợp với điều kiện tìm kiếm.
- Quản trị viên thực hiện xóa người dùng khi cần thiết.
- Hệ thống cập nhật danh sách người dùng.
- Quản trị viên truy cập chức năng quản lý hệ thống.
- Quản trị viên xem thông tin hệ thống.
- Hệ thống hiển thị các thông tin về trạng thái và tài nguyên hệ thống.
- Quản trị viên thực hiện dọn dẹp cache.
- Hệ thống tiến hành xóa các dữ liệu cache không cần thiết.
- Quản trị viên thực hiện sao lưu dữ liệu hệ thống.
- Hệ thống tạo bản sao lưu và lưu trữ dữ liệu thành công.

Luồng phụ:

- Không tìm thấy người dùng phù hợp với điều kiện tìm kiếm.
- Quản trị viên hủy thao tác xóa người dùng.
- Người dùng được chọn để xóa không tồn tại hoặc đã bị xóa trước đó.
- Quá trình dọn dẹp cache thất bại do lỗi hệ thống.
- Không đủ dung lượng lưu trữ để thực hiện sao lưu dữ liệu.
- Quá trình sao lưu bị gián đoạn do mất kết nối hoặc lỗi máy chủ.
- Quản trị viên không có quyền thực hiện một số thao tác quản trị.

Điều kiện tiên quyết:

- Quản trị viên đã đăng nhập vào hệ thống.
- Quản trị viên có quyền quản trị hệ thống.
- Hệ thống đang hoạt động bình thường.

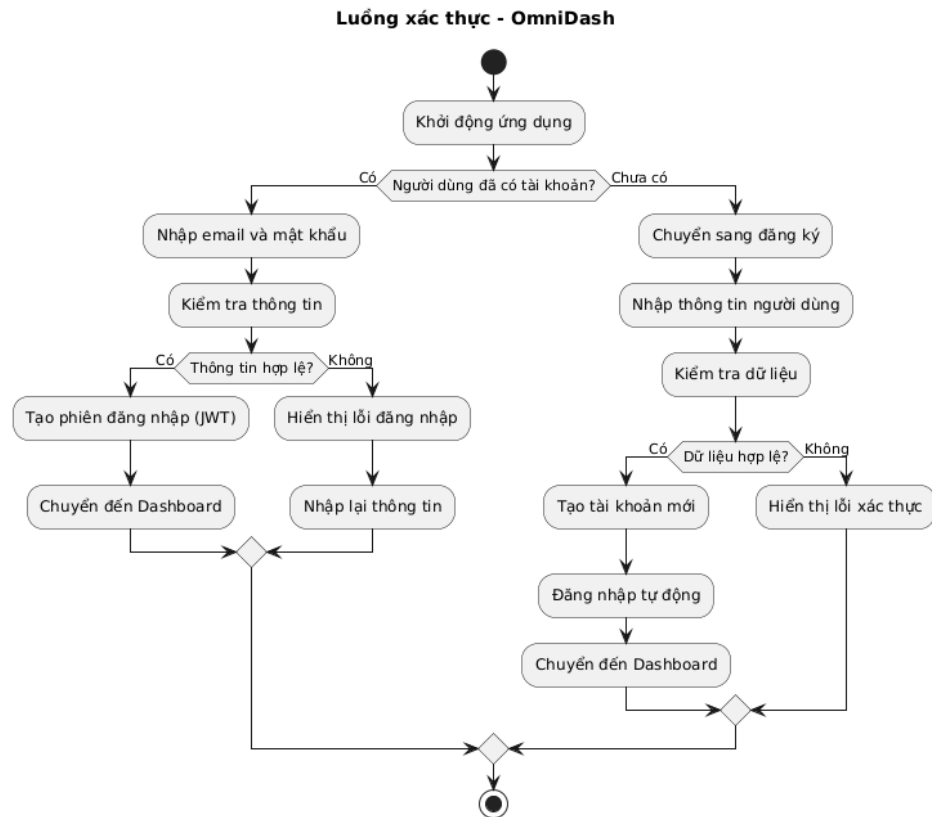
Điều kiện sau khi hoàn tất:

- Người dùng được xóa thành công khỏi hệ thống.
- Cache hệ thống được dọn dẹp thành công.
- Dữ liệu hệ thống được sao lưu thành công.
- Thông tin trạng thái hệ thống được cập nhật.

3.4 Các biểu đồ phân tích

3.4.1 Biểu đồ Activity Diagram

- Đăng ký/Đăng nhập

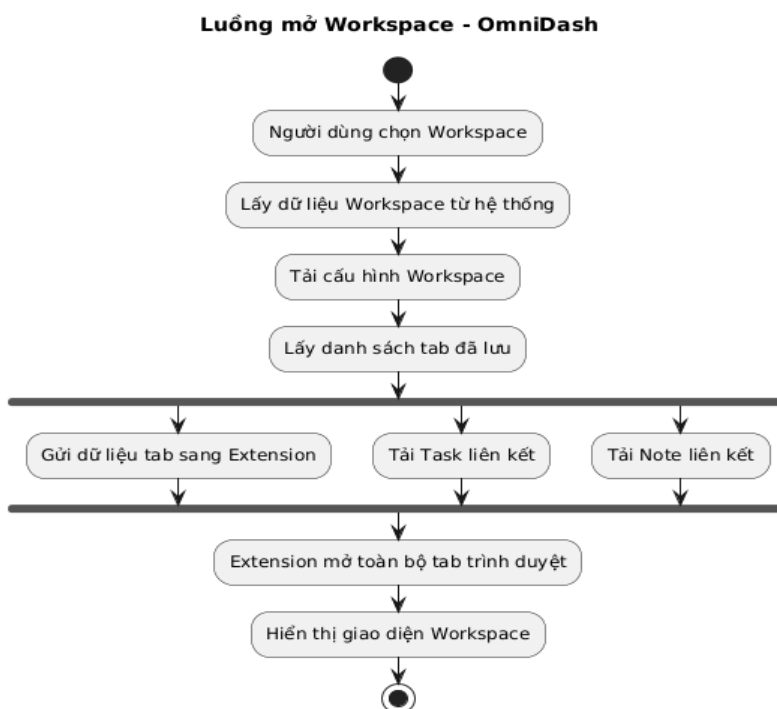


Hình 3.11: Activity Diagram Đăng ký/Đăng nhập

- Người dùng khởi động ứng dụng.
- Hệ thống kiểm tra xem người dùng đã có tài khoản hay chưa.
- Nếu đã có tài khoản, người dùng nhập email và mật khẩu.
- Hệ thống tiến hành kiểm tra thông tin đăng nhập.
- Nếu thông tin hợp lệ, hệ thống tạo phiên đăng nhập (JWT) và chuyển người dùng đến trang Dashboard.
- Nếu thông tin không hợp lệ, hệ thống hiển thị thông báo lỗi đăng nhập và yêu cầu người dùng nhập lại thông tin.

- Nếu người dùng chưa có tài khoản, hệ thống chuyển đến màn hình đăng ký.
- Người dùng nhập các thông tin cần thiết để tạo tài khoản.
- Hệ thống kiểm tra tính hợp lệ của dữ liệu đăng ký.
- Nếu dữ liệu hợp lệ, hệ thống tạo tài khoản mới và tự động đăng nhập cho người dùng.
- Sau khi đăng nhập thành công, hệ thống chuyển người dùng đến trang Dashboard.
- Nếu dữ liệu đăng ký không hợp lệ, hệ thống hiển thị thông báo lỗi xác thực và yêu cầu người dùng nhập lại thông tin.
- Quy trình kết thúc khi người dùng được chuyển đến Dashboard hoặc thoát khỏi luồng xác thực.

- **Mở Workspace**

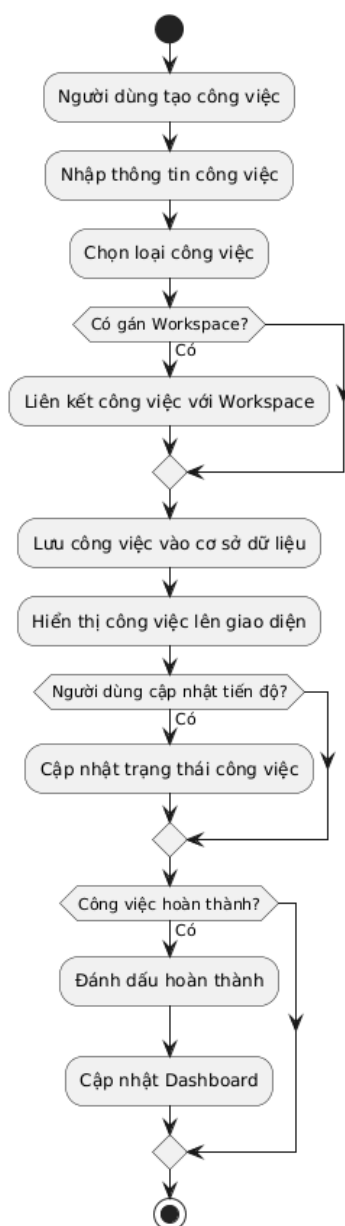


Hình 3.12: Activity Diagram mở workspace

- Người dùng chọn Workspace: Luồng hoạt động bắt đầu khi người dùng chọn một không gian làm việc cụ thể cần mở.
- Lấy dữ liệu Workspace từ hệ thống: Hệ thống truy xuất thông tin cơ bản của Workspace đang được yêu cầu.
- Tải cấu hình Workspace: Nạp các thiết lập và cấu hình tương ứng của Workspace đó.
- Lấy danh sách tab đã lưu: Trích xuất dữ liệu về các trang web đã được lưu trữ bên trong.
- Gửi dữ liệu tab sang Extension: Chuyển thông tin danh sách các tab cần mở cho tiện ích mở rộng (Extension) của trình duyệt (thực hiện đồng thời).
- Tải Task liên kết: Tải danh sách các công việc/nhiệm vụ được gắn với Workspace đó (thực hiện đồng thời).
- Tải Note liên kết: Tải các ghi chú thuộc về Workspace đó (thực hiện đồng thời).
- Extension mở toàn bộ tab trình duyệt: Sau khi nhận dữ liệu, tiện ích mở rộng tự động bật lên tất cả các tab trang web đã được chỉ định.
- Hiện thị giao diện Workspace: Quá trình kết thúc khi hệ thống hoàn tất tải và hiển thị giao diện không gian làm việc hoàn chỉnh cho người dùng.

- Quản lý công việc

Luồng quản lý công việc - OmniDash



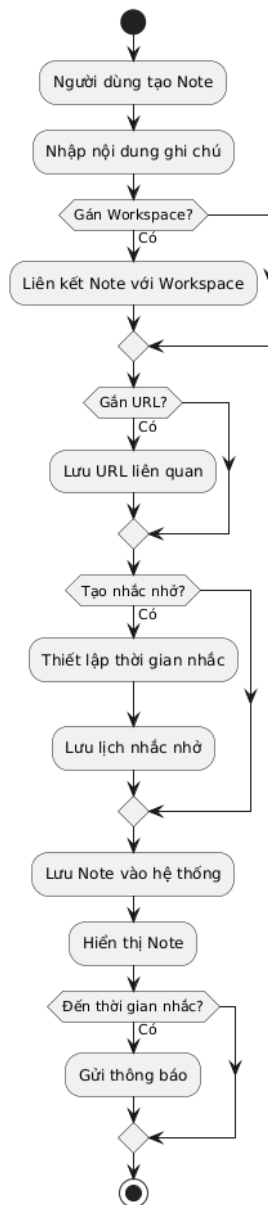
Hình 3.13: Activity Diagram quản lý công việc

- Người dùng tạo công việc: Luồng hoạt động bắt đầu khi người dùng chọn chức năng để khởi tạo một công việc mới trên hệ thống.

- Nhập thông tin công việc: Người dùng tiến hành điền các thông tin chi tiết, nội dung cần thiết cho công việc đó.
- Chọn loại công việc: Người dùng lựa chọn phân loại hoặc danh mục phù hợp cho công việc vừa tạo.
- Kiểm tra "Có gán Workspace?": Hệ thống kiểm tra xem công việc có được chỉ định vào một không gian làm việc nào không; nếu "Có", hệ thống tiến hành Liên kết công việc với Workspace, ngược lại sẽ bỏ qua bước này.
- Lưu công việc vào cơ sở dữ liệu: Hệ thống thực hiện lưu trữ toàn bộ thông tin của công việc vào cơ sở dữ liệu.
- Hiện thị công việc lên giao diện: Sau khi lưu thành công, công việc mới sẽ được cập nhật và hiển thị trực quan trên màn hình giao diện của người dùng.
- Kiểm tra "Người dùng cập nhật tiến độ?": Hệ thống ghi nhận hành vi của người dùng; nếu người dùng tiến hành cập nhật tiến trình, hệ thống sẽ thực hiện Cập nhật trạng thái công việc, ngược lại sẽ bỏ qua.
- Kiểm tra "Công việc hoàn thành?": Hệ thống xác định điều kiện xem công việc đã được làm xong chưa; nếu "Có", hệ thống sẽ chuyển sang Đánh dấu hoàn thành và tiến hành Cập nhật Dashboard để làm mới các số liệu thống kê, ngược lại luồng hoạt động sẽ đi thẳng tới điểm kết thúc.

- Quản lý ghi chú và nhắc nhở

Luồng Notes & Reminder - OmniDash

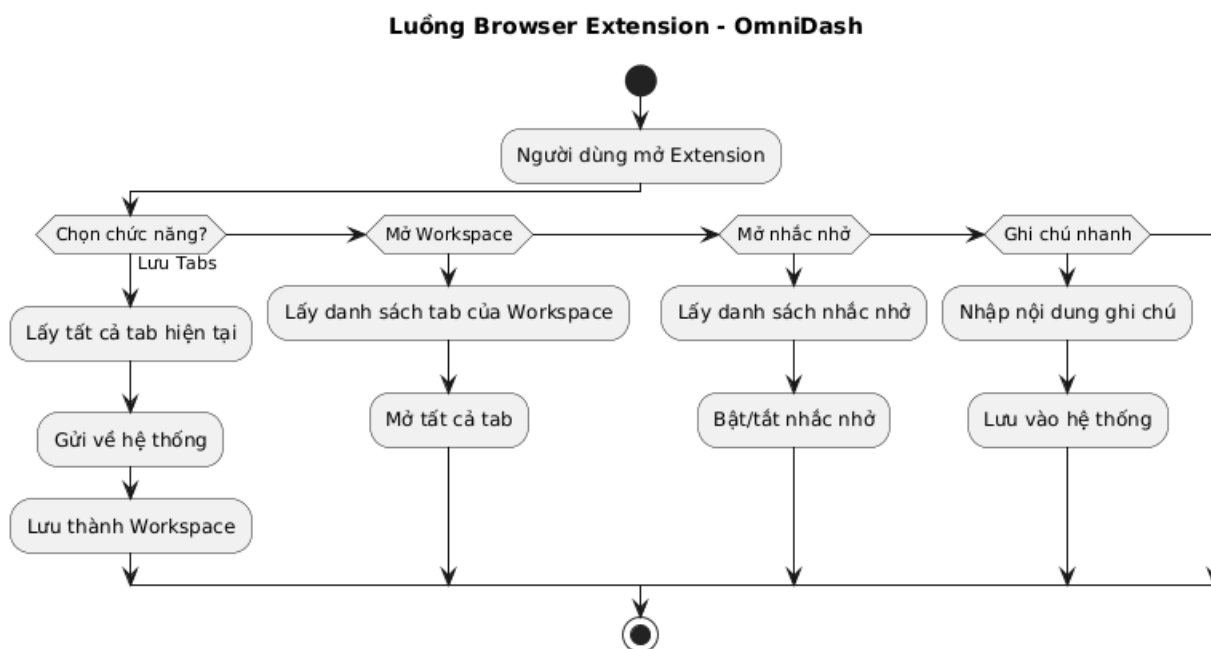


Hình 3.14: Activity Diagram quản lý bài tập

- Người dùng tạo Note: Bắt đầu luồng khi người dùng khởi tạo ghi chú mới.
- Nhập nội dung ghi chú: Người dùng điền thông tin chi tiết vào ghi chú.

- Kiểm tra "Gán Workspace?": Nếu có, hệ thống Liên kết Note với Workspace, ngược lại bỏ qua.
- Kiểm tra "Gắn URL?": Nếu có, tiến hành Lưu URL liên quan, ngược lại đi tiếp.
- Kiểm tra "Tạo nhắc nhở?": Nếu có, người dùng Thiết lập thời gian nhắc, sau đó hệ thống Lưu lịch nhắc nhở.
- Lưu Note vào hệ thống: Lưu toàn bộ dữ liệu ghi chú vào cơ sở dữ liệu.
- Hiển thị Note: Ghi chú mới được cập nhật và hiển thị lên giao diện.
- Kiểm tra "Đến thời gian nhắc?": Khi đến thời điểm đã hẹn, hệ thống sẽ Gửi thông báo cho người dùng, sau đó kết thúc luồng hoạt động.

- **Browser Extension**



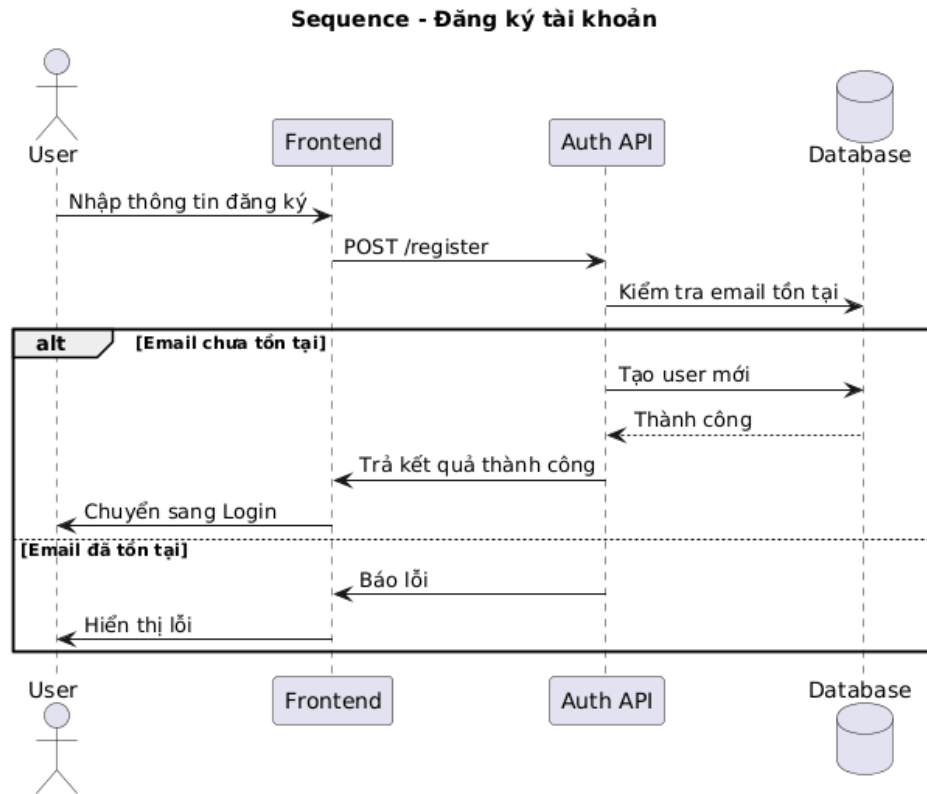
Hình 3.15: Activity Diagram Browser Extension

- Người dùng mở Extension: Bắt đầu luồng hoạt động khi tiện ích trên trình duyệt được bật.

- Chọn chức năng: Hệ thống cung cấp các lựa chọn và sẽ rẽ nhánh tùy thuộc vào quyết định của người dùng.
- Nhánh Lưu Tabs: Hệ thống Lấy tất cả tab hiện tại, Gửi về hệ thống và tiến hành Lưu thành Workspace.
- Nhánh Mở Workspace: Hệ thống Lấy danh sách tab của Workspace đã chọn và tự động Mở tất cả tab
- Nhánh Mở nhắc nhở: Hệ thống Lấy danh sách nhắc nhở để người dùng có thể Bật/tắt nhắc nhở
- Nhánh Ghi chú nhanh: Người dùng Nhập nội dung ghi chú, sau đó hệ thống sẽ Lưu vào hệ thống
- Tất cả các nhánh sau khi thực hiện xong thao tác đều hội tụ về điểm kết thúc luồng hoạt động.

3.4.2 Sequence Diagram

- Đăng ký



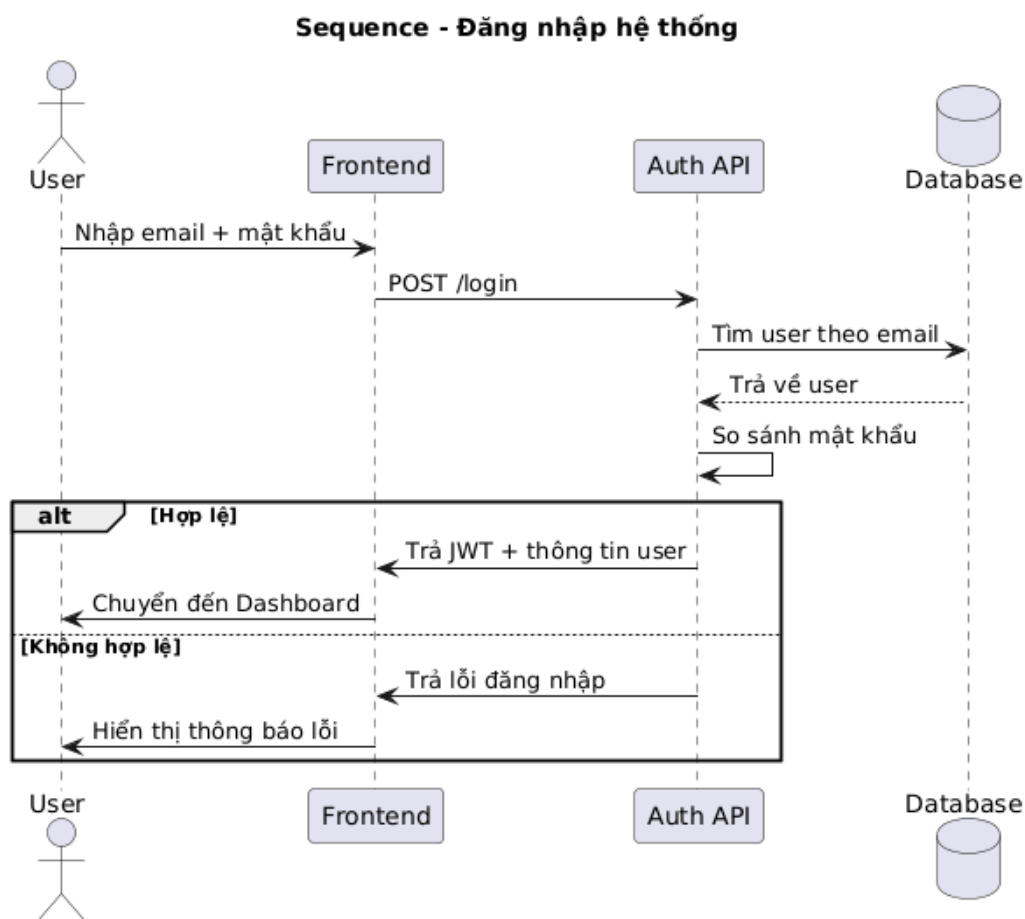
Hình 3.17: Sequence Diagram Đăng ký

- Nhập thông tin đăng ký: Người dùng (User) điền thông tin tài khoản vào giao diện (Frontend).
- Gửi yêu cầu đăng ký: Frontend gửi yêu cầu POST /register mang theo dữ liệu đến hệ thống xác thực (Auth API).
- Kiểm tra email tồn tại: Auth API gửi truy vấn đến Cơ sở dữ liệu (Database) để kiểm tra xem email này đã được sử dụng hay chưa.
- Trường hợp "Email chưa tồn tại": Auth API gửi lệnh yêu cầu Database tạo user mới. Khi Database phản hồi tạo thành công, Auth API trả kết quả tốt về

cho Frontend để tự động chuyển người dùng sang màn hình Đăng nhập (Login).

- Trường hợp "Email đã tồn tại": Auth API gửi phản hồi báo lỗi về cho Frontend, sau đó Frontend sẽ hiển thị thông báo lỗi trực tiếp lên màn hình cho người dùng.

- **Đăng nhập**

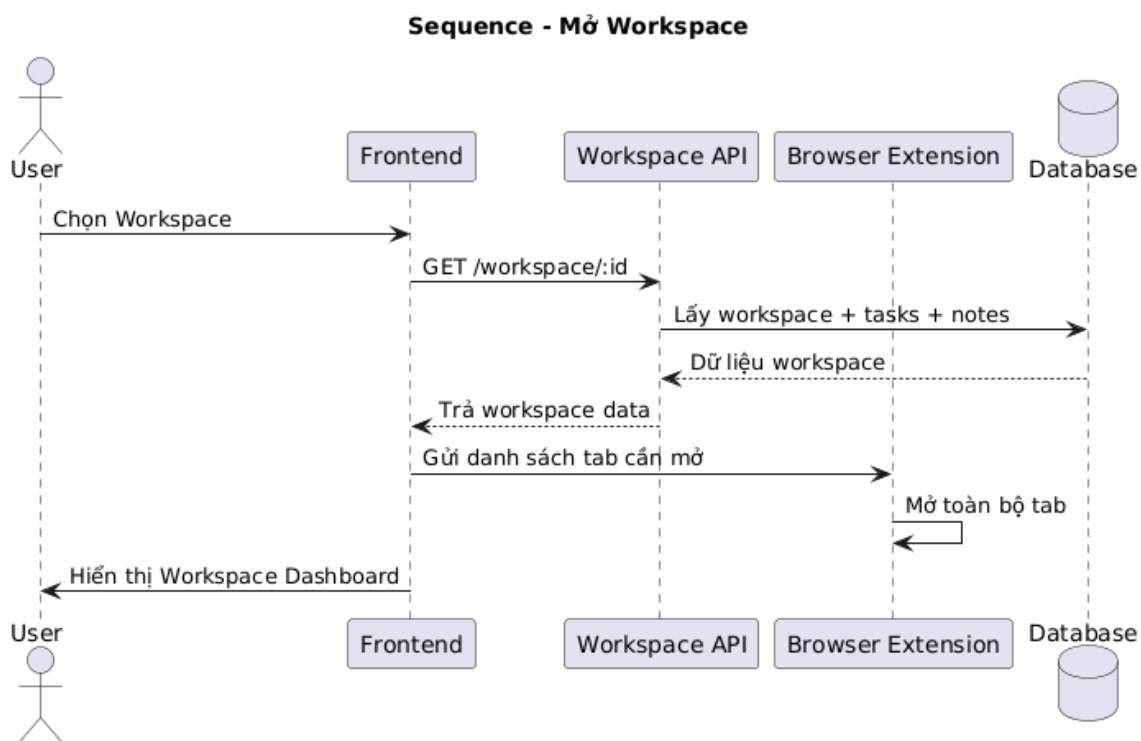


Hình 3.18: Sequence Diagram Đăng nhập

- Nhập thông tin: Người dùng điền email và mật khẩu vào giao diện (Frontend).

- Gửi yêu cầu: Frontend gọi API POST /login truyền dữ liệu tới hệ thống xác thực (Auth API).
- Xác thực: Auth API tìm kiếm người dùng trong Cơ sở dữ liệu (Database) dựa vào email, sau đó tiến hành so sánh mật khẩu.
- Trường hợp "Hợp lệ": Auth API cấp token (JWT) và thông tin user về cho Frontend để chuyển hướng người dùng thẳng vào trang Dashboard.
- Trường hợp "Không hợp lệ": Auth API phản hồi lỗi đăng nhập, Frontend tiếp nhận và hiển thị thông báo lỗi lên màn hình cho người dùng.

• Mở Workspace

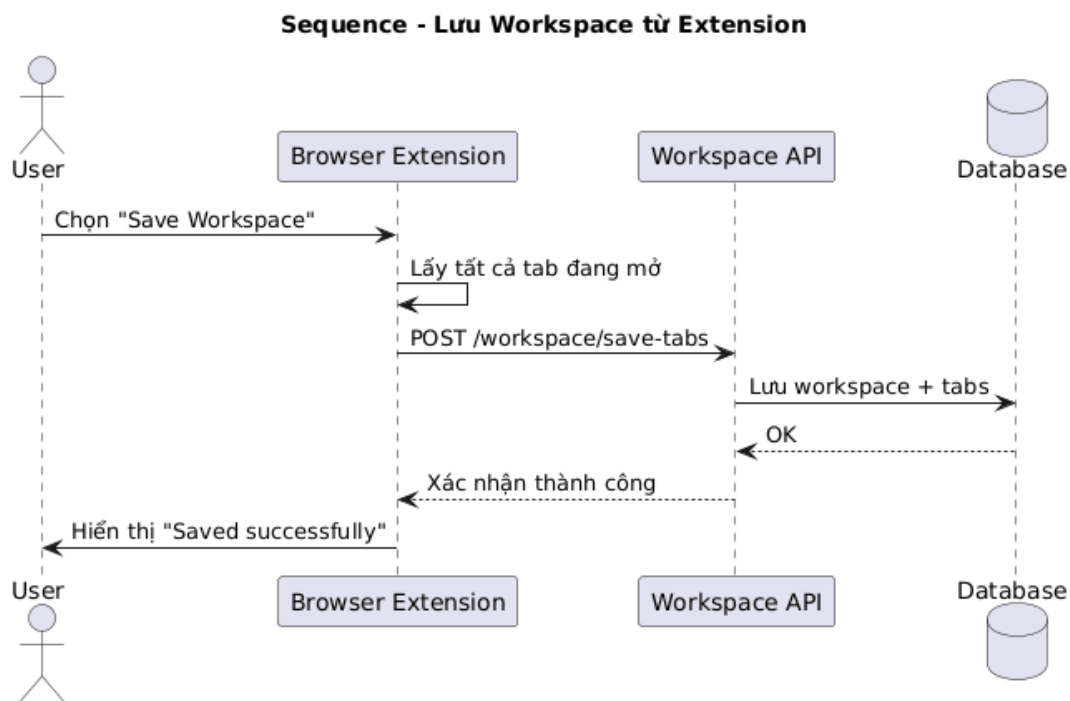


Hình 3.19: Sequence Diagram mở Workspace

- Chọn Workspace: Người dùng thao tác chọn một không gian làm việc trên giao diện (Frontend).

- Gửi yêu cầu: Frontend gọi API GET /workspace/:id đến hệ thống xử lý (Workspace API).
- Truy xuất dữ liệu: Workspace API lấy thông tin bao gồm workspace, tasks và notes từ Cơ sở dữ liệu (Database) rồi trả kết quả về cho Frontend.
- Mở tab: Frontend gửi tiếp danh sách các tab cần mở sang Tiện ích mở rộng (Browser Extension) để tiện ích tự động bật toàn bộ tab lên.
- Hiển thị: Cuối cùng, Frontend hoàn tất quá trình và hiển thị giao diện Workspace Dashboard cho người dùng.

- **Lưu workspace từ Extension**

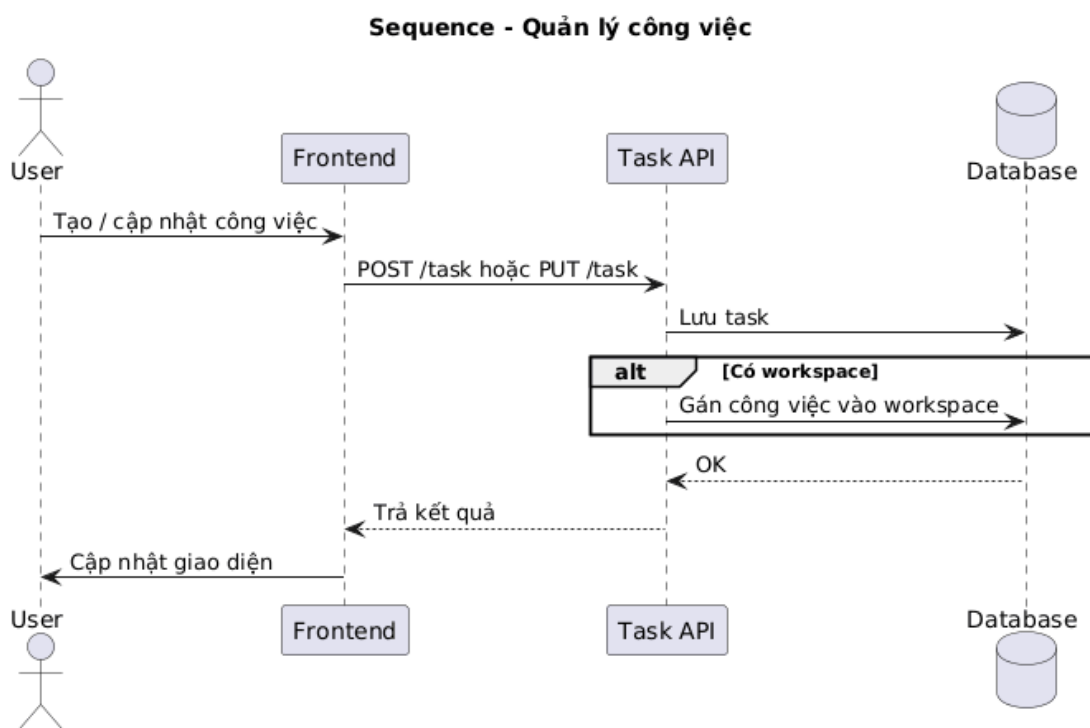


Hình 3.20: Sequence Diagram lưu workspace từ extension

- Thao tác lưu: Người dùng chọn chức năng "Save Workspace" trên Tiện ích mở rộng (Browser Extension).

- Lấy dữ liệu: Browser Extension tự động thu thập thông tin của tất cả các tab đang mở trên trình duyệt.
- Gửi yêu cầu: Browser Extension gọi API POST /workspace/save-tabs để đẩy dữ liệu về hệ thống (Workspace API).
- Lưu trữ: Workspace API tiếp nhận và tiến hành lưu thông tin workspace cùng danh sách tab vào Cơ sở dữ liệu (Database).
- Hoàn tất: Sau khi Database phản hồi lưu thành công (OK), Workspace API gửi xác nhận về cho Browser Extension để tiện ích này hiển thị thông báo "Saved successfully" tới người dùng.

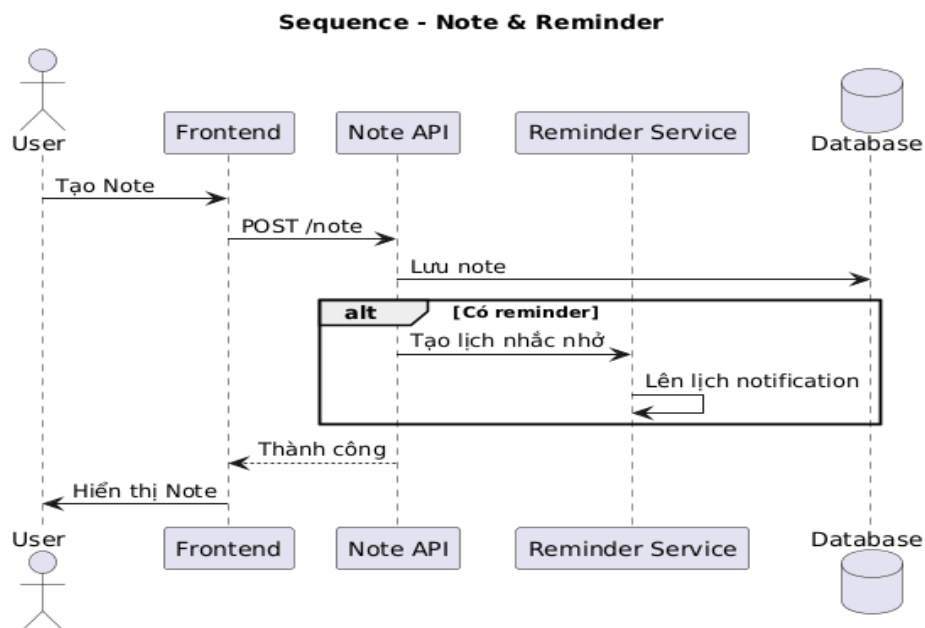
- **Quản lý công việc**



Hình 3.21: Sequence Diagram quản lý công việc

- Thao tác: Người dùng thực hiện lệnh tạo mới hoặc cập nhật một công việc trên giao diện (Frontend).
- Gửi yêu cầu: Frontend gọi API POST /task (tạo mới) hoặc PUT /task (cập nhật) để truyền dữ liệu đến hệ thống xử lý (Task API).
- Lưu trữ: Task API tiếp nhận yêu cầu và tiến hành lưu thông tin công việc vào Cơ sở dữ liệu (Database).
- Xử lý "Có workspace": Nếu công việc được đính kèm vào một không gian làm việc cụ thể, hệ thống sẽ thực hiện thêm lệnh gán công việc đó vào workspace tương ứng trong Database.
- Hoàn tất: Sau khi Database xác nhận lưu thành công (OK), Task API trả kết quả về cho Frontend để tiến hành cập nhật lại giao diện hiển thị cho người dùng.

- **Ghi chú và nhắc nhở**



Hình 3.22: Sequence Diagram ghi chú và nhắc nhở

- Tạo Note: Người dùng thao tác tạo ghi chú mới trên giao diện (Frontend).
- Gửi yêu cầu: Frontend gọi API POST /note để truyền dữ liệu đến hệ thống xử lý (Note API).
- Lưu trữ: Note API tiếp nhận và tiến hành lưu thông tin ghi chú vào Cơ sở dữ liệu (Database).
- Xử lý "Có reminder": Nếu ghi chú có cài đặt nhắc nhở, Note API sẽ gửi lệnh đến Dịch vụ nhắc nhở (Reminder Service) để hệ thống này tự động lên lịch thông báo.
- Hoàn tất: Sau khi xử lý xong, Note API trả kết quả thành công về Frontend để hiển thị ghi chú mới lên màn hình cho người dùng.

3.5 Kết luận chương

Trong chương III, đề tài đã tiến hành phân tích hệ thống dựa trên yêu cầu thực tế của người dùng và mục tiêu xây dựng hệ thống quản lý dự án học tập. Các nội dung chính bao gồm khảo sát hiện trạng, xác định yêu cầu chức năng và phi chức năng, xây dựng mô hình Use Case, Activity Diagram và Sequence Diagram nhằm mô tả chi tiết luồng xử lý nghiệp vụ của hệ thống.

Thông qua các mô hình phân tích, hệ thống đã được mô tả một cách trực quan về vai trò của các tác nhân, quá trình tương tác giữa người dùng và hệ thống cũng như các bước xử lý dữ liệu giữa giao diện, API và cơ sở dữ liệu. Việc phân tích này giúp làm rõ cấu trúc và logic hoạt động của hệ thống, đồng thời tạo nền tảng quan trọng cho quá trình thiết kế kiến trúc, cơ sở dữ liệu và triển khai hệ thống ở chương tiếp theo.

CHƯƠNG IV: THIẾT KẾ HỆ THỐNG

4.1 Kiến trúc tổng thể

4.1.1 Mô hình kiến trúc hệ thống

Hệ thống quản lý công việc đa ngữ cảnh OmniDash được xây dựng theo mô hình Kiến trúc phân lớp (Layered Monolithic Architecture) kết hợp với Tiện ích mở rộng trình duyệt (Browser Extension) nhằm đảm bảo tính mở rộng, linh hoạt và khả năng tự động hóa không gian làm việc.

Kiến trúc hệ thống gồm các tầng chính:

- Tầng trình bày (Presentation Layer/Frontend): Bao gồm hai thành phần chính:
 - Web Application: Được phát triển bằng Next.js (React) kết hợp Tailwind CSS và Zustand. Chịu trách nhiệm hiển thị giao diện tối giản, quản lý trạng thái ngữ cảnh (Work, Personal, Chill), và gửi yêu cầu đến máy chủ thông qua RESTful API.
 - Browser Extension: Được phát triển theo chuẩn Manifest V3 của Google, đóng vai trò tiếp nhận lệnh từ Web App để trực tiếp thao tác và điều khiển hệ thống Tab trình duyệt.
- Tầng xử lý nghiệp vụ (Business Logic/Backend): Được xây dựng bằng Node.js và ExpressJS. Tầng chịu trách nhiệm:
 - Xử lý logic nghiệp vụ cốt lõi (điều phối Workspace, lọc hiển thị Task).
 - Xác thực người dùng và bảo mật phiên hoạt động bằng JSON Web Token (JWT).
 - Đảm bảo tính nhất quán của dữ liệu trước khi đẩy xuống cơ sở dữ liệu.

- Tầng Truy xuất và Dữ liệu (Persistence & Data Layer): Sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu quan hệ (RDBMS). Dữ liệu được tổ chức chặt chẽ thông qua các bảng có ràng buộc quan hệ và được quản lý, thao tác thông qua công cụ Prisma ORM.

4.1.2 Nguyên tắc thiết kế

Hệ thống OmniDash được thiết kế dựa trên các nguyên tắc cốt lõi:

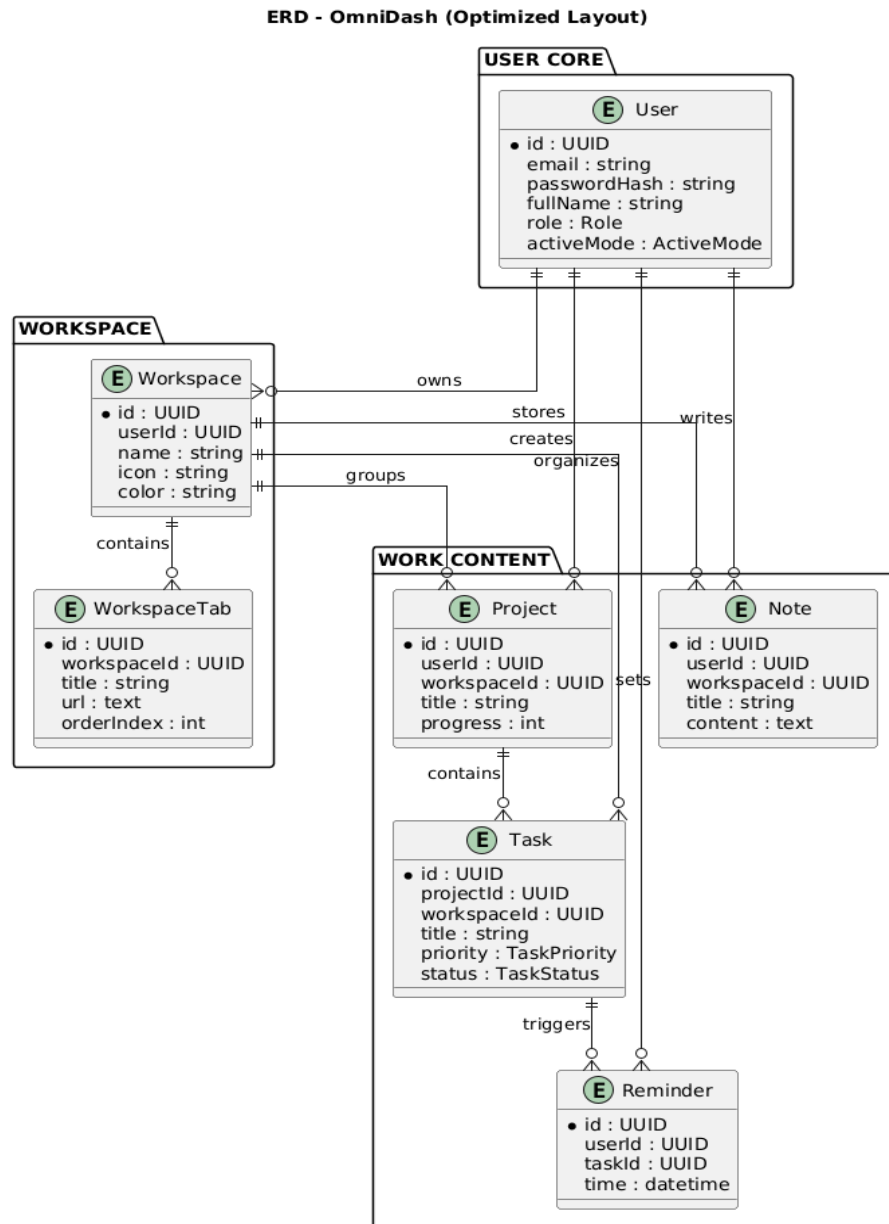
- Phân tách trách nhiệm (Separation of Concerns): Tuân thủ quy tắc phụ thuộc một chiều từ trên xuống giữa các lớp để dễ dàng bảo trì và mở rộng (Extensibility).
- Tính tích hợp (Integrability): Đảm bảo luồng giao tiếp bảo mật và liền mạch giữa Web Application và Browser Extension thông qua cơ chế Message Passing.
- Đảm bảo tính bảo mật: Mật khẩu người dùng được băm (hashing) bằng bcrypt, và các phiên hoạt động được bảo vệ an toàn thông qua xác thực Token.
- Tối ưu hiệu năng: Cấu trúc API và truy vấn cơ sở dữ liệu (qua Prisma) được tối ưu hóa để đảm bảo thời gian phản hồi khi kích hoạt "Workspace Combos" luôn diễn ra mượt mà dưới 1.5 giây.

4.2 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng MongoDB với mô hình document-based. Các thực thể chính và mối quan hệ được thiết kế nhằm đảm bảo phản ánh đúng nghiệp vụ quản lý lớp học và đồ án.

4.2.1 Mô hình ERD (Entity Relationship Diagram)

Mô hình ERD mô tả các thực thể (entities) chính trong hệ thống quản lý công việc và mối quan hệ giữa chúng. Dữ liệu được lưu trữ trong PostgreSQL dưới dạng document, tuy nhiên vẫn có thể biểu diễn dưới dạng ERD để thể hiện cấu trúc logic hệ thống.



Hình 4.1: Sơ đồ ERD

4.2.2 Các bảng dữ liệu chính

- **User**

Bảng 4.1: Bảng dữ liệu User

Trường	Kiểu dữ liệu	Mô tả
Id	UUID	Khóa chính
email	String	Email đăng nhập (duy nhất)
passwordHash	String	Mật khẩu đã mã hóa
fullName	String	Họ tên người dùng
role	Enum	Vai trò người dùng
activeMode	Enum	Chế độ hoạt động hiện tại

- **Workspace**

Bảng 4.2: Bảng dữ liệu Workspace

Trường	Kiểu dữ liệu	Mô tả
Id	UUID	Khóa chính
userId	UUID	Tham chiếu User
Name	String	Tên không gian làm việc
Icon	String	Biểu tượng Workspace
Color	String	Màu sắc hiển thị

- **WorkspaceTab**

Bảng 4.3: Bảng dữ liệu WorkspaceTab

Trường	Kiểu dữ liệu	Mô tả
Id	UUID	Khóa chính
workspaceId	UUID	Tham chiếu Workspace
Title	String	Tên tab
url	Text	Đường dẫn trang web
orderIndex	Int	Thứ tự hiển thị tab

- **Project**

Bảng 4.4: Bảng dữ liệu Project

Trường	Kiểu dữ liệu	Mô tả
Id	UUID	Khóa chính
userId	UUID	Tham chiếu User
workspaceId	UUID	Tham chiếu Workspace
Title	String	Tên dự án
Progress	Int	Tiến độ hoàn thành(%)

- **Task**

Bảng 4.5: Bảng dữ liệu Task

Trường	Kiểu dữ liệu	Mô tả
Id	UUID	Khóa chính
projectId	UUID	Tham chiếu Project
workspaceId	UUID	Tham chiếu Workspace
Title	String	Tên nhiệm vụ
Priority	Enum	Mức độ ưu tiên
Status	Enum	Trạng thái nhiệm vụ

- **Note**

Bảng 4.6: Bảng dữ liệu Note

Trường	Kiểu dữ liệu	Mô tả
Id	UUID	Khóa chính
userId	UUID	Tham chiếu User
workspaceId	UUID	Tham chiếu workspace
Title	String	Tiêu đề ghi chú
Content	Text	Nội dung ghi chú

- **Reminder**

Bảng 4.7: Bảng dữ liệu Reminder

Trường	Kiểu dữ liệu	Mô tả
Id	UUID	Khóa chính
userId	UUID	Tham chiếu User
taskId	UUID	Tham chiếu Task
Time	Datetime	Thời gian nhắc nhở

4.3 Thiết kế thành phần phần mềm

Hệ thống quản lý công việc OmniDash được thiết kế theo mô hình Kiến trúc nguyên khối phân lớp (Layered Monolithic Architecture). Trong kiến trúc này, toàn bộ mã nguồn Backend được triển khai thành một khối ứng dụng duy nhất nhưng được phân tách thành các lớp (layers) có trách nhiệm rõ ràng, kết hợp với tầng Frontend và Browser Extension để tạo nên một hệ thống hoàn chỉnh, dễ bảo trì và phát triển.

4.3.1 Thành phần giao diện người dùng

Frontend của ứng dụng Web được xây dựng bằng Next.js (React) kết hợp với TypeScript, Tailwind CSS và Zustand, đảm nhiệm các chức năng:

- Hiển thị giao diện và tiếp nhận các thao tác từ người dùng.
- Quản lý trạng thái ứng dụng (lưu trữ danh sách công việc, không gian làm việc đang hiển thị).
- Gửi yêu cầu dữ liệu đến Backend thông qua RESTful API.

- Phát tín hiệu truyền tin (Message Passing) chứa danh sách URL sang Tiện ích mở rộng (Browser Extension) để thực thi tự động mở tab.

Các module giao diện chính gồm:

- Module xác thực người dùng (Đăng nhập, Đăng ký).
- Module bảng điều khiển tổng quan (Dashboard).
- Module quản lý công việc (Task Management: Thêm, sửa, xóa, đánh dấu hoàn thành công việc).
- Module quản lý không gian làm việc (Workspace Combos: Nhóm các URL tài liệu/trang web cần thiết cho công việc).

4.3.2 Thành phần xử lý nghiệp vụ

Backend được xây dựng bằng Node.js và ExpressJS, chịu trách nhiệm xử lý logic nghiệp vụ và quản lý dữ liệu.

Các thành phần chính:

- API Layer (Tầng giao tiếp): Tiếp nhận HTTP request từ Frontend và trả về dữ liệu định dạng JSON.
- Controller Layer (Tầng điều khiển): Kiểm tra tính hợp lệ của dữ liệu đầu vào (DTO) và điều phối luồng xử lý.
- Service Layer (Tầng nghiệp vụ): Chứa logic xử lý chính (ví dụ: tạo và chỉnh sửa Workspace, gom nhóm danh sách URL tài nguyên, quản lý các thao tác CRUD của công việc).
- Middleware: Chịu trách nhiệm xác thực mã thông báo (JWT) và bảo vệ các API nội bộ của hệ thống.

4.3.3 Thành phần dữ liệu

Hệ thống sử dụng cơ sở dữ liệu quan hệ PostgreSQL kết hợp với công cụ Prisma ORM.

Chức năng chính:

- Lưu trữ tập trung dữ liệu người dùng, không gian làm việc (Workspace), tài nguyên (Resource URLs), và danh sách công việc (Task).
- Đảm bảo tính toàn vẹn dữ liệu thông qua các ràng buộc khóa ngoại chặt chẽ.
- Tối ưu hóa các truy vấn thông qua cơ chế ánh xạ thực thể của Prisma Client.

4.3.4 Thành phần giao tiếp và thực thi trình duyệt

Khác với các ứng dụng web thông thường, OmniDash có thêm một thành phần hoạt động độc lập trên trình duyệt (chuẩn Manifest V3), đảm nhiệm:

- Chạy ngầm tệp lệnh background.js (Service Worker) để liên tục lắng nghe các thông điệp từ Web App.
- Tiếp nhận mảng dữ liệu URLs được Web App gửi sang khi người dùng bấm nút khởi tạo một Workspace.
- Sử dụng API hệ thống của trình duyệt (chrome.tabs.create) để tự động mở hàng loạt trang web tương ứng với không gian làm việc đã chọn.

4.3.5 Mối liên kết giữa các thành phần

- Frontend (Next.js) nhận thao tác của người dùng và gửi request đến Backend thông qua REST API.

- Tại Backend, request đi qua Middleware để xác thực JWT, sau đó Controller gọi Service để xử lý logic.
- Service sử dụng Prisma để tương tác với Database (PostgreSQL) và lấy ra dữ liệu cần thiết
- Kết quả JSON được Backend trả về cho Frontend.
- Frontend sử dụng cơ chế truyền tin (PostMessage) gửi mảng URL sang cho Extension.
- Extension gọi API trình duyệt để tự động mở các Tab tài nguyên, thiết lập không gian làm việc cho người dùng.

4.4 Thiết kế giao diện người dùng

4.4.1 Nguyên tắc thiết kế

Giao diện hệ thống được thiết kế dựa trên các nguyên tắc:

- Đơn giản, dễ sử dụng.
- Bố cục rõ ràng, trực quan.
- Đồng nhất màu sắc và thành phần UI.
- Tối ưu trải nghiệm người dùng.
- Hỗ trợ responsive trên nhiều kích thước màn hình.

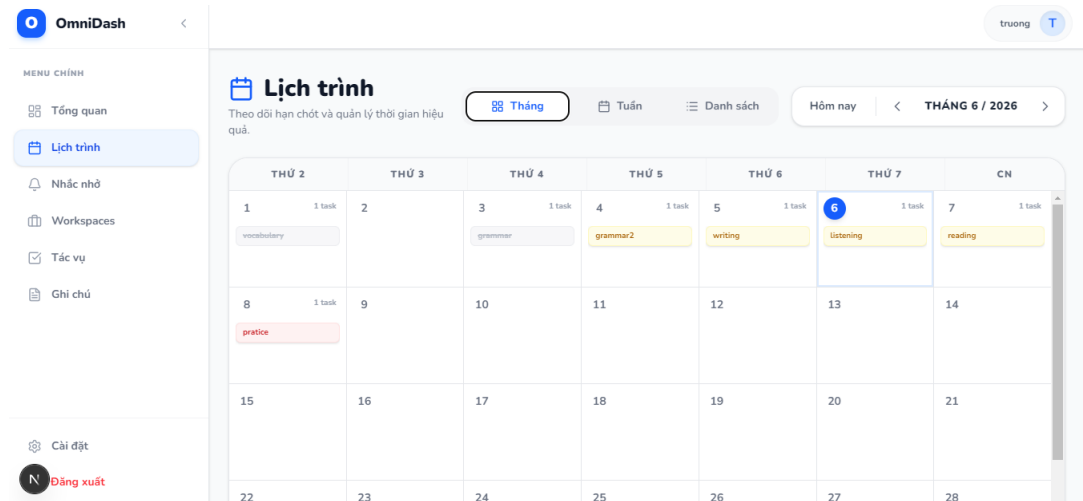
4.4.2 Các màn hình chính

- **Màn hình Dashboard**



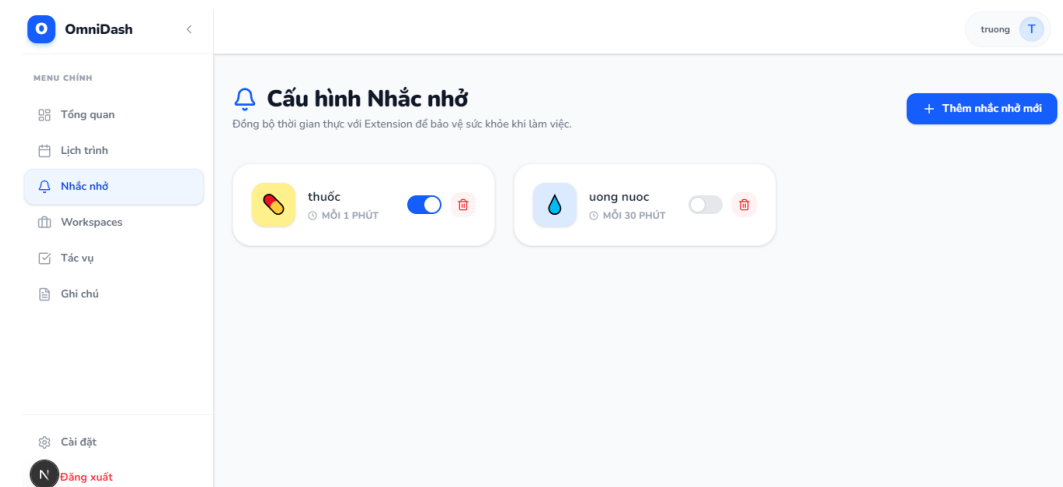
Hình 4.2: Giao diện Dashboard

- **Màn hình lịch trình**



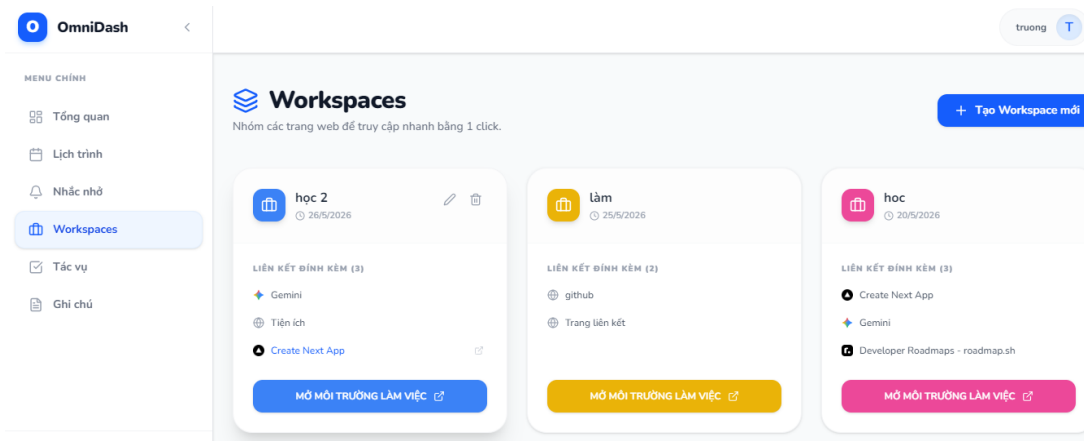
Hình 4.3: Giao diện Lịch trình

- **Màn hình nhắc nhở**



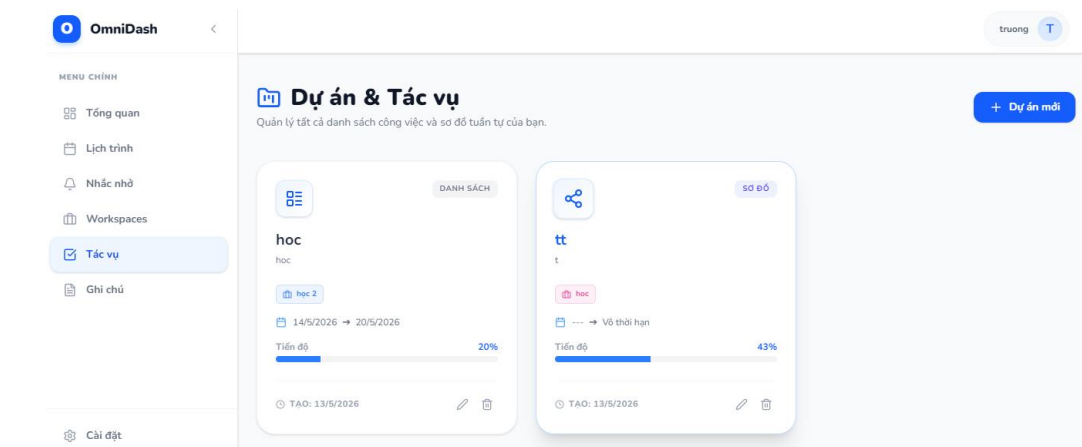
Hình 4.4: Giao diện nhắc nhở

- **Màn hình workspace**



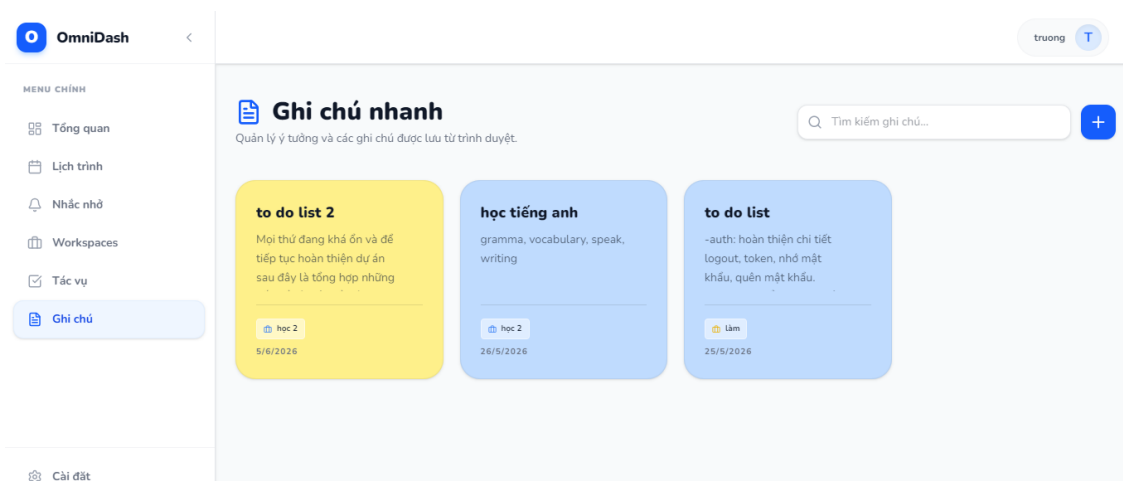
Hình 4.5: Giao diện quản lý workspace

- **Màn hình công việc**



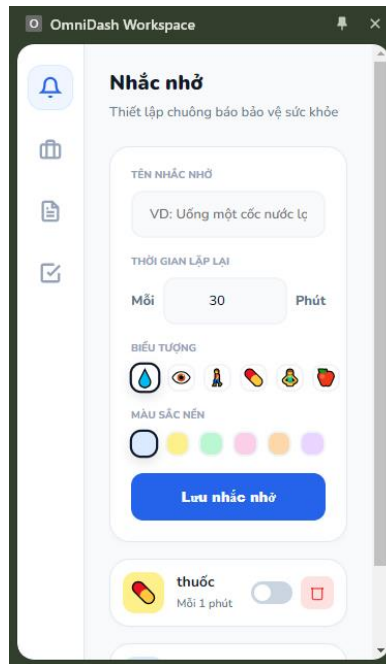
Hình 4.6: Giao diện quản lý công việc

- **Màn hình ghi chú**



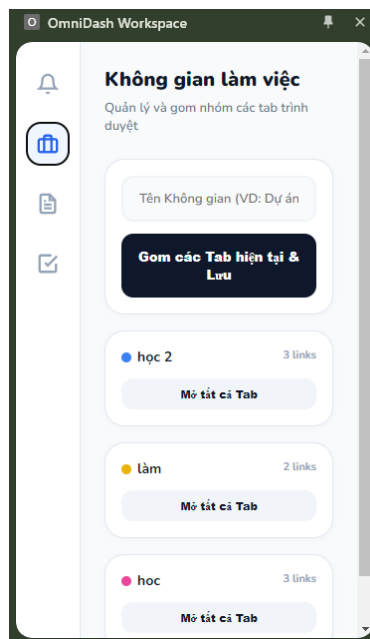
Hình 4.7: Giao diện ghi chú

- **Màn hình nhắc nhở ở extension**



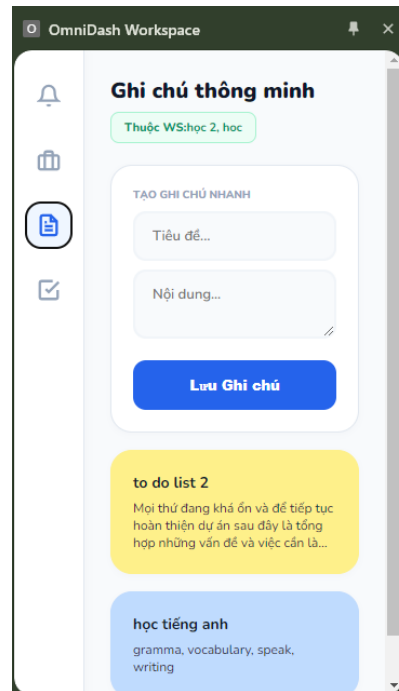
Hình 4.8: Giao diện nhắc nhở ở extension

- **Màn hình workspace ở extension**



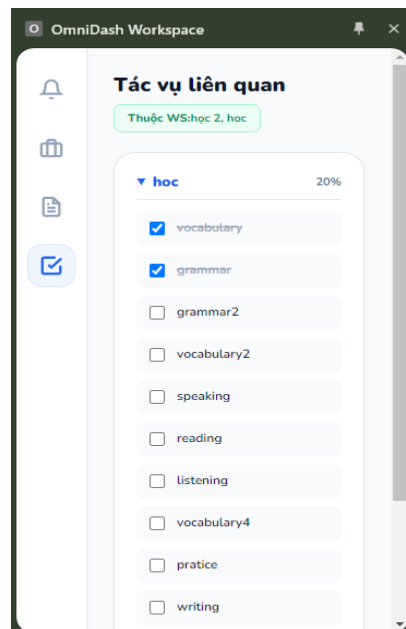
Hình 4.9: Giao diện workspace ở extension

- **Màn hình workspace ở extension**



Hình 4.10: Giao diện ghi chú ở extension

- **Màn hình công việc ở extension**



Hình 4.11: Giao diện công việc ở extension

4.5 Thiết kế xử lý

Phần này mô tả các luồng xử lý chính của hệ thống.

4.5.1 *Luồng xử lý đăng nhập*

- Người dùng nhập email và mật khẩu.
- Frontend (Next.js) gửi request đến API đăng nhập.
- Backend (Express.js) kiểm tra thông tin đối chiếu trong database (PostgreSQL).
- Nếu hợp lệ, hệ thống trả về JWT.
- Frontend lưu token (vào cookie/local storage) và chuyển hướng đến trang dashboard.

4.5.2 *Luồng xử lý tạo workspace*

- Người dùng nhập tên không gian làm việc và danh sách các đường dẫn web (URL) tài liệu cần thiết.
- Frontend gửi request POST tạo Workspace.
- Backend xác thực dữ liệu đầu vào và lưu dữ liệu vào database thông qua Prisma.
- Hệ thống trả về kết quả thành công và giao diện tự động cập nhật danh sách Workspace mới.

4.5.3 *Luồng xử lý quản lý công việc*

- Người dùng nhập thông tin công việc mới (tên công việc, hạn chót) và có thể tùy chọn gắn với một Workspace.
- Frontend gửi request thêm/sửa công việc đến hệ thống API.

- Backend tiếp nhận, xử lý logic và cập nhật dữ liệu vào database.
- Hệ thống trả về kết quả, danh sách công việc trên màn hình được cập nhật ngay lập tức.

4.5.4 Luồng xử lý kích hoạt workspace

- Người dùng nhấn nút "Kích hoạt" tại một Workspace cụ thể trên giao diện.
- Frontend gọi API để lấy mảng danh sách các URL thuộc về Workspace đó.
- Backend truy vấn database và trả về danh sách dữ liệu.
- Frontend phát tín hiệu truyền tin (Message Passing) gửi danh sách URL này sang Tiện ích mở rộng (Browser Extension).
- Extension tiếp nhận dữ liệu và dùng API của trình duyệt để tự động mở đồng loạt các tab web.

4.5.5 Luồng xử lý cập nhật trạng thái công việc

- Người dùng đánh dấu hoàn thành (tick) một công việc trên giao diện.
- Frontend gửi request cập nhật trạng thái (PATCH/PUT) đến Backend.
- Backend xử lý và cập nhật trạng thái công việc thành "Hoàn thành" trong database.
- Hệ thống trả về xác nhận, giao diện lập tức phản hồi (gạch bỏ chữ hoặc chuyển công việc xuống mục đã hoàn thành).

4.6 Kết luận chương

Chương IV đã trình bày thiết kế tổng thể của hệ thống từ kiến trúc phần mềm, cơ sở dữ liệu, các thành phần chức năng cho đến thiết kế giao diện và xử lý nghiệp vụ. Việc thiết kế theo mô hình phân lớp giúp hệ thống dễ mở rộng, bảo trì và đáp

ứng tốt yêu cầu thực tế của bài toán quản lý công việc. Những nội dung trong chương này là cơ sở quan trọng để triển khai và kiểm thử hệ thống trong chương tiếp theo.

CHƯƠNG V: KIỂM THỬ HỆ THỐNG

5.1 Mục tiêu kiểm thử

Mục tiêu của quá trình kiểm thử là đảm bảo hệ thống quản lý công việc OmniDash hoạt động đúng theo các yêu cầu đã phân tích và thiết kế, đồng thời đảm bảo tính ổn định, bảo mật và khả năng đáp ứng luồng tương tác giữa ứng dụng Web và tiện ích mở rộng trên trình duyệt.

Các mục tiêu cụ thể gồm:

- Kiểm tra tính chính xác của các chức năng chính (đăng nhập, quản lý không gian làm việc, quản lý công việc, tự động hóa mở tab).
- Phát hiện và khắc phục lỗi trong quá trình xử lý dữ liệu từ Backend xuống cơ sở dữ liệu PostgreSQL.
- Đảm bảo hệ thống xác thực người dùng an toàn bằng JWT và bảo vệ dữ liệu cá nhân của từng tài khoản.
- Kiểm tra tính ổn định của kênh truyền tin (Message Passing) giữa giao diện Web và Browser Extension.
- Đảm bảo thời gian phản hồi của hệ thống từ lúc bấm nút đến lúc trình duyệt mở tab đạt hiệu suất cao.

5.2 Chiến lược kiểm thử

Hệ thống được kiểm thử theo các cấp độ sau:

5.2.1 Kiểm thử đơn vị

- Kiểm tra từng chức năng riêng lẻ trong backend như:
 - Xác thực người dùng (Đăng nhập/Đăng ký)
 - Thêm, sửa, xóa Workspace và danh sách URL
 - Thêm và cập nhật trạng thái Task
- Đảm bảo từng endpoint API trả về đúng cấu trúc dữ liệu JSON và mã trạng thái HTTP (200, 400, 401)

5.2.2 Kiểm thử tích hợp

- Kiểm tra sự tương tác giữa tầng Frontend (Next.js) và tầng Backend (Express.js).
- Kiểm tra luồng giao tiếp giữa Web Application và Browser Extension thông qua các sự kiện trình duyệt.
- Đảm bảo dữ liệu được truyền chính xác giữa các tầng và xác thực JWT hoạt động đúng đắn khi gửi request.

5.2.3 Kiểm thử hệ thống

- Kiểm tra toàn bộ hệ thống hoạt động như một chỉnh thể.
- Mô phỏng các tình huống thực tế:
 - Người dùng đăng nhập vào hệ thống.
 - Tạo mới một Workspace kèm theo các URL tài nguyên.
 - Thêm danh sách công việc tương ứng.

- Nhấn kích hoạt Workspace để Extension mở hàng loạt tab làm việc.

5.2.4 Kiểm thử chấp nhận

- Cho người dùng thử nghiệm hệ thống trực tiếp trên trình duyệt Chrome.
- Ghi nhận phản hồi về trải nghiệm UI/UX (độ mượt mà, phản hồi báo lỗi) và điều chỉnh giao diện nếu cần

5.3 Môi trường kiểm thử

Quá trình kiểm thử được thực hiện trong môi trường sau:

- Hệ điều hành: Windows 10
- Trình duyệt: Google Chrome
- Node.js: phiên bản LTS
- Hệ quản trị cơ sở dữ liệu: PostgreSQL.
- Công cụ hỗ trợ:
 - Postman (kiểm thử API)
 - Prisma Studio hoặc pgAdmin (quản lý và kiểm tra dữ liệu trực tiếp).

Hệ thống được triển khai ở môi trường local để đánh giá chức năng và khả năng phản hồi kỹ thuật của Extension trước khi đóng gói phát hành.

5.4 Kịch bản kiểm thử

5.4.1 Kiểm thử chức năng đăng nhập

Bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi
Nhập email, mật khẩu đúng	Email hợp lệ, mật khẩu đúng	Đăng nhập thành công, cấp JWT, chuyển Dashboard
Nhập sai mật khẩu	Email đúng, mật khẩu sai	Hiển thị thông báo lỗi
Nhập email không tồn tại	Email sai	Thông báo tài khoản không tồn tại

Bảng 5.1: Kiểm thử đăng nhập

5.4.2 Kiểm thử tạo không gian làm việc(workspace)

Bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi
Người dùng tạo Workspace mới	Tên Workspace hợp lệ, kèm mãng URL đúng định dạng	Workspace được tạo thành công, lưu vào database, hiển thị trên UI
Bỏ trống tên Workspace	Tên rỗng, URL hợp lệ	Hệ thống chặn thao tác, báo lỗi yêu cầu nhập tên
Nhập URL sai định dạng	Tên hợp lệ, chuỗi URL thiếu http:// hoặc https://	Backend báo lỗi validation (400 Bad Request)

Bảng 5.2: Kiểm thử tạo lớp học

5.4.3 Kiểm thử chức năng quản lý công việc

Bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi
Thêm công việc mới	Tên công việc (Title) hợp lệ	Công việc hiển thị vào danh sách hiện tại
Cập nhật trạng thái công việc	Click vào checkbox "Hoàn thành"	Gọi API PATCH thành công, UI gạch ngang tên công việc
Xóa công việc	Nhấn icon xóa (Delete) tại một Task	Xóa bản ghi trong database, mất khỏi danh sách hiển thị

Bảng 5.3: Kiểm thử quản lý công việc

5.4.4 Kiểm thử tương tác Web và Extension

Bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi
Nhấn nút "Activate" trên UI	Extension đã được cài đặt và bật	Extension nhận thông điệp, tự động mở tất cả URL trong tab mới
Nhấn nút "Activate" trên UI	Extension chưa cài đặt hoặc bị tắt	Trình duyệt không có phản hồi mở tab, UI hiện cảnh báo kiểm tra Extension
Đo thời gian phản hồi	Kích hoạt Workspace chứa 5 URLs	Các tab bắt đầu mở trong thời gian dưới 1.5 giây

Bảng 5.4: Kiểm thử tương tác web và extension

5.4.5 Kiểm thử bảo mật luồng API

Bước thực hiện	Hành động	Kết quả mong đợi
Gọi API lấy danh sách Task	Request có chứa JWT Token hợp lệ ở Header	Trả về đúng danh sách công việc của user đó (HTTP 200)
Gọi API lấy danh sách Task	Không có Token hoặc Token bị chỉnh sửa	Middleware chặn lại, trả về lỗi HTTP 401 (Unauthorized)
Lấy Workspace của user khác	Truyền workspaceId không thuộc quyền sở hữu	Tầng Service chặn lại, trả về lỗi HTTP 403 (Forbidden)

Bảng 5.5: Kiểm thử bảo mật luồng API

5.5 Kết luận chương

Chương V đã trình bày mục tiêu, chiến lược và các kịch bản kiểm thử của hệ thống. Quá trình kiểm thử cho thấy các chức năng chính hoạt động đúng theo yêu cầu, đảm bảo tính ổn định và đáp ứng tốt nhu cầu quản lý công việc và các chức năng khác. Những lỗi phát sinh trong quá trình kiểm thử đã được ghi nhận và khắc phục kịp thời, góp phần nâng cao chất lượng hệ thống trước khi triển khai chính thức.

CHƯƠNG VI: CÀI ĐẶT VÀ KẾT QUẢ

6.1 Môi trường triển khai

Hệ thống quản lý công việc OmniDash được triển khai trong môi trường phát triển và thử nghiệm với các công nghệ và công cụ sau:

6.1.1 Phần mềm

- Hệ điều hành: Windows 10
- Trình duyệt: Google Chrome (chạy Extension ở chế độ Developer Mode)
- Node.js: phiên bản LTS
- Quản lý dữ liệu: Prisma Studio / pgAdmin
- Visual Studio Code
- Git & GitHub
- Postman (kiểm thử API)
- Hệ quản trị CSDL: PostgreSQL

6.1.2 Công nghệ sử dụng

- Frontend: Next.js (React) + TypeScript + Tailwind CSS + Zustand
- Backend: Node.js + ExpressJS
- Database: PostgreSQL kết hợp Prisma ORM
- Browser Extension: Manifest V3 (Chrome Extension API)
- Authentication: JWT

6.2 Cài đặt các chức năng chính

6.2.1 Cài đặt chức năng xác thực

- Người dùng đăng ký và đăng nhập tài khoản bằng email.
- Backend mã hóa mật khẩu bằng thư viện bcrypt trước khi lưu vào CSDL.
- Sau khi xác thực thành công, hệ thống sinh ra JWT token để quản lý quyền truy cập cho các phiên làm việc tiếp theo.

6.2.2 Cài đặt chức năng quản lý không gian làm việc (Workspace)

- Người dùng tạo các Workspace và gắn các đường dẫn tài nguyên (URLs) cần thiết cho công việc.
- Backend tiếp nhận dữ liệu, kiểm tra tính hợp lệ và sử dụng Prisma Client để lưu trữ thông tin có ràng buộc khóa ngoại vào PostgreSQL.

6.2.3 Cài đặt chức năng quản lý công việc

- Người dùng có thể tạo, chỉnh sửa, xóa và cập nhật trạng thái các công việc.
- Cho phép người dùng liên kết một công việc cụ thể vào một Workspace đã tạo.
- Giao diện cập nhật danh sách công việc ngay lập tức khi có thao tác thay đổi trạng thái thành "Đã hoàn thành".

6.2.4 Cài đặt chức năng tự động hóa trình duyệt (Browser Extension)

- Khởi tạo tệp background.js (Service Worker) trên tiện ích mở rộng để lắng nghe sự kiện.
- Khi người dùng nhấn "Kích hoạt" trên Web, Frontend sử dụng window.postMessage gửi mảng URL sang Extension.

- Extension sử dụng API `chrome.tabs.create` để tự động mở hàng loạt trang web tương ứng, thiết lập sẵn sàng môi trường làm việc cho người dùng.

6.3 Kết quả thực nghiệm

Sau quá trình triển khai và kiểm thử, hệ thống đạt được các kết quả sau:

- Các chức năng quản lý (User, Task, Workspace) hoạt động ổn định, dữ liệu đảm bảo tính toàn vẹn.
- Thời gian phản hồi API từ Backend (Monolith) nhanh chóng trong môi trường thử nghiệm.
- Luồng giao tiếp giữa Web App và Browser Extension hoạt động mượt mà, thao tác mở hàng loạt tab đạt độ trễ thấp (dưới 1.5 giây).
- Cấu trúc cơ sở dữ liệu với PostgreSQL và Prisma hoạt động chính xác, không xảy ra lỗi truy vấn.
- Giao diện hiển thị trực quan, tối giản, thân thiện với người dùng.

6.4 Đánh giá hệ thống

6.4.1 Ưu điểm

- Kiến trúc Layered Monolith phân tách rõ ràng các tầng giao tiếp, nghiệp vụ và dữ liệu, giúp dễ dàng bảo trì và mở rộng sau này.
- Tính năng "Workspace Combos" kết hợp Extension giúp giải quyết triệt để bài toán thiết lập môi trường làm việc thủ công, tiết kiệm thời gian cho người dùng.
- Cơ chế bảo mật tốt với JWT và bcrypt.
- Giao diện UI/UX tối giản, loại bỏ sự xao nhãng.

6.4.2 Hạn chế

- Tiện ích mở rộng hiện tại mới chỉ hỗ trợ các trình duyệt sử dụng nhân Chromium (Chrome, Edge, Brave), chưa hỗ trợ Safari hoặc Firefox.
- Hệ thống mới hoạt động trên môi trường cục bộ (Localhost) và máy chủ thử nghiệm, chưa triển khai thực tế lên nền tảng đám mây (Cloud).
- Chưa có tính năng đồng bộ hóa và nhận thông báo đẩy (Push Notification) qua thiết bị di động.

6.4.3 Hướng cải tiến

- Triển khai hệ thống (Deploy) lên các dịch vụ Cloud như Vercel (Frontend), Render/AWS (Backend) và Supabase (PostgreSQL).
- Đóng gói và phát hành tiện ích mở rộng lên Cửa hàng Chrome trực tuyến (Chrome Web Store).
- Bổ sung trang thống kê tiến độ công việc bằng các biểu đồ trực quan (Chart.js / Recharts) để người dùng theo dõi hiệu suất cá nhân.
- Phát triển thêm tính năng chia sẻ Workspace cho người dùng khác (phục vụ làm việc nhóm).

TÀI LIỆU THAM KHẢO

1. Tài liệu hướng dẫn chính thức của Next.js, *Next.js Documentation*. Truy cập tại:
<https://nextjs.org/docs>
2. Tài liệu phát triển tiện ích mở rộng trình duyệt, *Chrome Extensions Documentation (Manifest V3)*. Truy cập tại:
<https://developer.chrome.com/docs/extensions/mv3/>
3. Tài liệu hệ quản trị cơ sở dữ liệu và ORM, *Prisma Documentation*. Truy cập tại:
<https://www.prisma.io/docs>